Genomic Insights into Walnut Blight: A Workbook

Aidan C. Shands

2025-03-05

Contents

1 Preface **5**

2 Introduction **7**
2.1 Hypotheses 7

3 Methods **9**
3.1 Read Processing 9
3.2 Genome Assembly and Annotation 9
3.3 *In Vitro* Copper Sensistivity 21
3.4 Pangenome Analyses 21
3.5 Phylogenetic Analyses 22
3.6 Type III Effectors 28

4 Results **31**
4.1 Read Processing 31
4.2 Genome Assembly and Annotation 34
4.3 *In Vitro* Copper Sensitivity 36
4.4 Pangenome Analyses (NEED TO FINISH) 36
4.5 Phylogenetic Analyses (NEED TO FINISH) 43

5 Scripts **45**
5.1 Read Processing 45
5.2 De Novo Genome Assembly 46
5.3 Pangenome 60
5.4 Phylogenetic Analyses 61
5.5 Molecular Clock 75
5.6 Type III Effectors 83

Chapter 1

Preface

This is a digital laboratory notebook that I used for my postdoctoral research in the lab of Dr. James E. Adaskaveg at University of California, Riverside.

Enjoy.



Figure 1.1: Walnut blight symptoms. Credit: James E. Adaskaveg and Helga Förster

Chapter 2

Introduction

The objective of this research is to characterize the population structure and prevalence of copper resistance of the walnut blight pathogen, *Xanthomonas arboricola* pv. *juglandis* (*Xaj*) collected in central and Northern California between the years of 1994 to 2019. Additionally, we are interested in elucidating the type III secretion system effector (T3E) diversity and assessing where the Californian *Xaj* population fits in a global context. We also sequenced 20 strains of *Xanthomonas arboricola* pv. *pruni* (*Xap*), the causal agent of bacterial spot of almond, and included those in some of the analyses.

2.1 Hypotheses

- Genes associated with and/or responsible for copper resistance are prevalent in the CA *Xanthomonas arboricola* pv. *juglandis* population and correlate with the respective copper-sensitivity phenotype *in vitro*.
- *Xaj* and *Xap* exhibit variation in type III secretion system effector repertoires.
- CA *Xaj* strains form distinct genetic clusters when compared with other global strains.
- The CA *Xaj* population does not encode any plasmids.

Chapter 3

Methods

3.1 Read Processing

Whole-genome sequencing from Novogene generated high-quality reads (Table 4.1). Novogene provides reads that have been filtered based on the following criteria:

(1) Remove reads containing adapters.

(2) Remove reads containing N > 10% (N represents the base cannot be determined).

(3) Remove reads containing low quality (Qscore<= 5) base which is over 50% of the total base.

However, I performed an additional filtering step using FastP v0.23.4 (Chen et al. (2018)). The filters I'm applying are as follows:

```
-q 30 (the quality value that a base is qualified)
--detect_adapter_for_pe (automatically detect adapter sequence based on paired end reads, and rem
```

Since we received ~2x the amount of sequencing data than we were quoted for, I chose to filter the data for the highest quality reads (q $\geq$ 30). I also opted to enable the auto-detection for adapter removal. Novogene removes most adapter sequences but < 1% of the read sets contain adapter sequences so FastP *should* detect and remove the remainders. Please refer to FastP.sh (5.1.1) for the script.

3.2 Genome Assembly and Annotation

Prior to settling on the pipeline detailed below, I tested several other pipelines that are detailed in `XAJ_XAP_Project.v1.md`, `AS_Progress_Report_090924.v3.pptx`, and `Assembly_Pipeline_102424.pptx`. Based on the findings from the pipeline

tests, we settled on the following steps for *de novo* genome assembly and annotation (3.1):

1. *De novo* assembly using Platanus_B v1.3.2 (Kajitani et al. (2020)).
2. Generate assembly stats via QUAST v5.1.0rc1 (Gurevich et al. (2013)).
3. Scaffold contigs using Ragout v2.3 (Kolmogorov et al. (2014)).
    - Xaj isolate IVIA 1317 (NCBI Accession: GCF_023375035.1) (Cuesta-Morrondo et al. (2022)) was used as the reference for the Xaj population.
    - Xap isolate 15-088 (NCBI Accession: GCF_008761555.1) was used as the reference for the Xap population.
4. Polish scaffolds with Pilon (Walker et al. (2014)).
5. Assess completeness and contamination with CheckM (Parks et al. (2015)).
6. Annotate with NCBI PGAP (Tatusova et al. (2016)).
    - Since I already generated the Bioproject and Biosamples I was able to generate NCBI-submission ready annotations.
    - These are currently being processed by NCBI. Two samples had an error and I'm in communication with NCBI on how to resolve these errors.

3.2.1 Steps 1-2. Platanus_B & QUAST

I generated `Assembly_Workflow_Platanus_B.csv` and used it as my `$SAMPFILE` variable for running Platanus_B with the Step1.Assemble.sh (5.2.1) script. It contains 5 columns that detail the sample ID, Strain, filtered Read1, filtered Read2, and organism. The `Assembly_Workflow_Platanus_B.csv` file was converted to unix format using the `dos2unix Assembly_Workflow_Platanus_B.csv` command on the cluster and afterwards the nano `Assembly_Workflow_Platanus_B.csv` command was used to add one blank line at the end. Step1.Assemble.sh (5.2.1) generated the de novo assemblies (contigs) and QUAST was used to generate assembly statistics.

I wrote a series of scripts to summarize the annotation and assembly results. These scripts are:

Summarize_ANI.sh (5.2.2), Summarize_CheckM.sh (5.2.3), and Summarize_Genbank.py (5.2.4)

3.2.2 Step 3. Scaffolding

`Ragout v2.3` was used to scaffold the assemblies using Step2.Scaffold.sh (5.2.6) . I used Xaj 417 and IVIA 1317 as reference strains for Xaj and 15-088 as the reference strain for Xap. As mentioned previously, we opted to proceed with the Ragout scaffolds using the IVIA 1317 reference since it gave better results (see `Assembly_Pipeline_102424.pptx`). QUAST was also used to generate statistics for comparisons.

Ragout uses `.rcp` files as inputs to set specific variables. I generated two

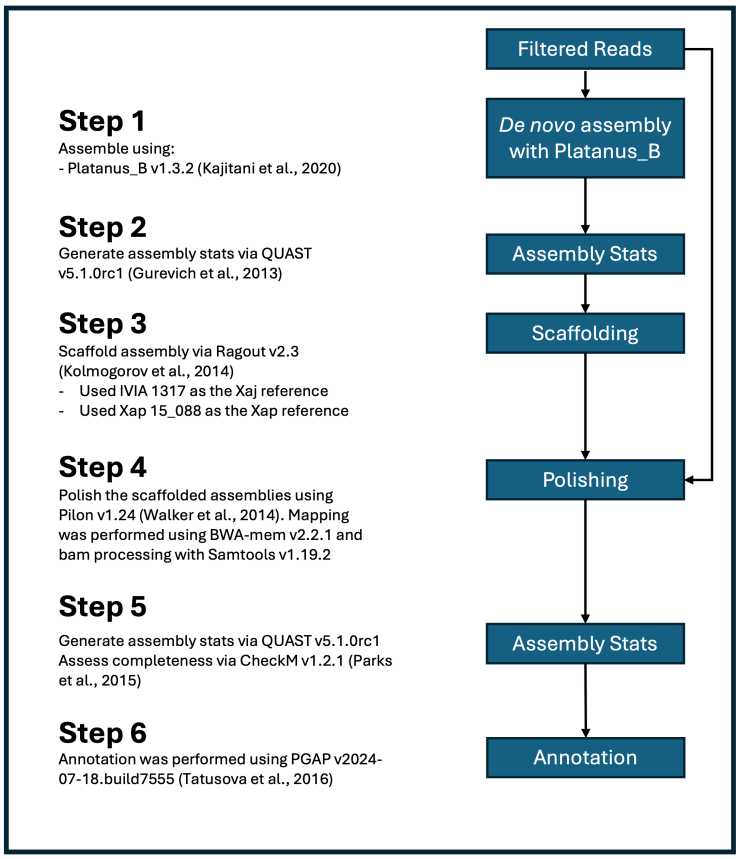


Figure 3.1: Xaj and Xap assembly pipeline

template `.rcp` files for Xaj (`XAJ_Template.rcp`) and Xap (`XAP_Template.rcp`), respectively. These template files served as an input for `.rcp` file generation for each strain. `XAJ_Strains.RCP.csv` and `XAP_Strains.RCP.csv` served as input files for generating the `.rcp` files using Generate_RCP.sh (5.2.5).

3.2.3 Step 4. Polishing with Pilon

Based on the the previous findings (see `Assembly_Pipeline_102424.pptx`), Pilon increased assembly quality. `Pilon v1.24` using default settings were applied to the Ragout assemblies using Step3.Polish.sh (5.2.7). I used `BWA-mem2 v2.2.1` (Vasimuddin et al. (2019)), and `samtools v1.19.2` (Li et al. (2009)) prior to Pilon. These Pilon polished assemblies were used for the PGAP annotations. The Pilon-polished assemblies were then subjected to CheckM (Parks et al. (2015)) using the taxonomy option with *Xanthomonas* as the reference genus to assess completeness and contamination (see Step3.CheckM.sh (5.2.8)).

3.2.4 Step 5. Annotation

I downloaded `PGAP v2024-07-18.build7555` on my local computer because I was running into issues trying to install it on the cluster. Since these assemblies and annotations will be submitted to NCBI, I needed to generate `$STRAIN.generic.yaml` and `$STRAIN.submol.yaml` as inputs for PGAP to ensure that all metadata is correctly assigned. I wrote Generate_Generic_YAML.sh (5.2.9) and Generate_Submol_YAML.sh (5.2.10) to efficiently generate the .yaml files. The metadata was stored in `Reference_Data_For_YAML.txt` and each of the .yaml scripts used that file to generate the respective .yaml files. Next I wrote PGAP.sh (5.2.11), the core PGAP command, to generate the annotations and I called that script using Step4.Annotate.sh (5.2.12) because I was running this locally.

3.2.5 NCBI Submission

Before we publish the manuscript we must submit our data (raw Illumina reads & annotated assemblies) to NCBI. I submitted all the raw reads from Novogene to the SRA under the BioProject PRJNA1151720 (3.1). The SRA submission has already been processed and will be released upon publication. The genomes are currently being processed as of January 17,2025.

Unfortunately, XAJ2128 and XAJ934 had one error each that I must address before submission. The error is the same for both: `[SEQ_FEAT.FeatureBeginsOrEndsInGap] Feature begins or ends in gap` and only occurs for one gene (transposase) in each strain, respectively. Specifically, the beginning of the genes start with an 'N', but the respective protein started with an 'M'. According to NCBI, the fix is to either remove the gene entirely or label the gene as 'partial'. Since I've already performed genomic analyses (phylogeny, pangenome, etc.) I do not want to remove the gene. I emailed NCBI on January 21, 2025 to ask for assistance to make these changes to indicate that the transposase is indicated as 'partial'. I'm

confident that they will be able to help me make these changes. This is likely due to me using the `--ignore-all-errors` argument in PGAP.sh (5.2.11). I had to use that argument in order to make the program run.. otherwise the annotations would fail. It's fine as the other annotations are processing just fine.

Here are the BioProject details (please note that this information can be modified if needed, as I wrote these as placeholders):

BioProject Accession:

- PRJNA1151720

Title:

- Population genomics of *Xanthomonas arboricola* pv. *juglandis* and *X. arboricola* pv. *pruni* from California

Description:

- *Xanthomonas arboricola* pv. *juglandis* (*Xaj*) and *X. arboricola* pv. *pruni* (*Xap*) are the causal agents of walnut blight and bacterial spot of almond, respectively. California is the global leader in almond production and the top producer of walnuts in the United States, but both crops face challenges from *Xaj* and *Xap*, respectively. Moreover, the persistent use of copper-based bactericides has led to copper-resistant strains of *Xaj* and *Xap*. In this study, we sequenced and assembled the genomes of 120 *Xaj* and 20 *Xap* strains isolated from walnut and almond in Central and Northern California, respectively, to determine their population structure, detect copper resistance within the populations, and characterize their type III secretion system effector repertoires.

Table 3.1: List of all Xaj and Xap strains with SRA information.

Strain	BioSample	BioProject	Study	SRA Accession
XAJ1028	SAMN43323323	PRJNA1151720	SRP528351	SRR30358100
XAJ1046	SAMN43323324	PRJNA1151720	SRP528351	SRR30358099
XAJ1048	SAMN43323325	PRJNA1151720	SRP528351	SRR30358014
XAJ1049	SAMN43323326	PRJNA1151720	SRP528351	SRR30358003
XAJ1054	SAMN43323327	PRJNA1151720	SRP528351	SRR30357970
XAJ1082	SAMN43323328	PRJNA1151720	SRP528351	SRR30358043
XAJ1086	SAMN43323329	PRJNA1151720	SRP528351	SRR30358035
XAJ1148	SAMN43323330	PRJNA1151720	SRP528351	SRR30358000
XAJ1149	SAMN43323331	PRJNA1151720	SRP528351	SRR30357988
XAJ1197	SAMN43323332	PRJNA1151720	SRP528351	SRR30358075
XAJ1213	SAMN43323333	PRJNA1151720	SRP528351	SRR30358098
XAJ1239	SAMN43323334	PRJNA1151720	SRP528351	SRR30358087
XAJ1246	SAMN43323335	PRJNA1151720	SRP528351	SRR30358063
XAJ1250	SAMN43323336	PRJNA1151720	SRP528351	SRR30358052
XAJ1395	SAMN43323337	PRJNA1151720	SRP528351	SRR30358019

Strain	BioSample	BioProject	Study	SRA Accession
XAJ1399	SAMN43323338	PRJNA1151720	SRP528351	SRR30358018
XAJ145	SAMN43323339	PRJNA1151720	SRP528351	SRR30358021
XAJ1498	SAMN43323340	PRJNA1151720	SRP528351	SRR30358017
XAJ1535	SAMN43323341	PRJNA1151720	SRP528351	SRR30358016
XAJ1537	SAMN43323342	PRJNA1151720	SRP528351	SRR30358015
XAJ1539	SAMN43323343	PRJNA1151720	SRP528351	SRR30358013
XAJ1551	SAMN43323344	PRJNA1151720	SRP528351	SRR30358012
XAJ1554	SAMN43323345	PRJNA1151720	SRP528351	SRR30358011
XAJ1556	SAMN43323346	PRJNA1151720	SRP528351	SRR30358010
XAJ1557	SAMN43323347	PRJNA1151720	SRP528351	SRR30358009
XAJ1558	SAMN43323348	PRJNA1151720	SRP528351	SRR30358008
XAJ1595	SAMN43323349	PRJNA1151720	SRP528351	SRR30358007
XAJ1606	SAMN43323350	PRJNA1151720	SRP528351	SRR30358006
XAJ1613	SAMN43323351	PRJNA1151720	SRP528351	SRR30358005
XAJ1614	SAMN43323352	PRJNA1151720	SRP528351	SRR30358004
XAJ1615	SAMN43323353	PRJNA1151720	SRP528351	SRR30357980
XAJ1616	SAMN43323354	PRJNA1151720	SRP528351	SRR30357979
XAJ1617	SAMN43323355	PRJNA1151720	SRP528351	SRR30357978
XAJ1626	SAMN43323356	PRJNA1151720	SRP528351	SRR30357977
XAJ163	SAMN43323357	PRJNA1151720	SRP528351	SRR30357976
XAJ165	SAMN43323358	PRJNA1151720	SRP528351	SRR30357975
XAJ169	SAMN43323359	PRJNA1151720	SRP528351	SRR30357974
XAJ1710	SAMN43323360	PRJNA1151720	SRP528351	SRR30357973
XAJ1711	SAMN43323361	PRJNA1151720	SRP528351	SRR30357972
XAJ1808	SAMN43323362	PRJNA1151720	SRP528351	SRR30357971
XAJ181	SAMN43323363	PRJNA1151720	SRP528351	SRR30357969
XAJ1822	SAMN43323364	PRJNA1151720	SRP528351	SRR30357968
XAJ1824	SAMN43323365	PRJNA1151720	SRP528351	SRR30357967
XAJ2122	SAMN43323366	PRJNA1151720	SRP528351	SRR30357966
XAJ2124	SAMN43323367	PRJNA1151720	SRP528351	SRR30357965
XAJ2125	SAMN43323368	PRJNA1151720	SRP528351	SRR30357964
XAJ2128	SAMN43323369	PRJNA1151720	SRP528351	SRR30357963
XAJ2129	SAMN43323370	PRJNA1151720	SRP528351	SRR30357962
XAJ2131	SAMN43323371	PRJNA1151720	SRP528351	SRR30357961
XAJ2165	SAMN43323372	PRJNA1151720	SRP528351	SRR30358044
XAJ2167	SAMN43323373	PRJNA1151720	SRP528351	SRR30358042
XAJ2169	SAMN43323374	PRJNA1151720	SRP528351	SRR30358041
XAJ2170	SAMN43323375	PRJNA1151720	SRP528351	SRR30358040
XAJ2178	SAMN43323376	PRJNA1151720	SRP528351	SRR30358039
XAJ2180	SAMN43323377	PRJNA1151720	SRP528351	SRR30358038
XAJ283A	SAMN43323378	PRJNA1151720	SRP528351	SRR30358037
XAJ283B	SAMN43323379	PRJNA1151720	SRP528351	SRR30358036
XAJ30	SAMN43323380	PRJNA1151720	SRP528351	SRR30358034
XAJ31	SAMN43323381	PRJNA1151720	SRP528351	SRR30358033

Strain	BioSample	BioProject	Study	SRA Accession
XAJ32	SAMN43323382	PRJNA1151720	SRP528351	SRR30358032
XAJ321	SAMN43323383	PRJNA1151720	SRP528351	SRR30358031
XAJ322	SAMN43323384	PRJNA1151720	SRP528351	SRR30358030
XAJ33	SAMN43323385	PRJNA1151720	SRP528351	SRR30358029
XAJ330	SAMN43323386	PRJNA1151720	SRP528351	SRR30358028
XAJ333	SAMN43323387	PRJNA1151720	SRP528351	SRR30358026
XAJ338	SAMN43323388	PRJNA1151720	SRP528351	SRR30358025
XAJ34	SAMN43323389	PRJNA1151720	SRP528351	SRR30358024
XAJ340	SAMN43323390	PRJNA1151720	SRP528351	SRR30358027
XAJ341	SAMN43323391	PRJNA1151720	SRP528351	SRR30358002
XAJ344	SAMN43323392	PRJNA1151720	SRP528351	SRR30358001
XAJ345	SAMN43323393	PRJNA1151720	SRP528351	SRR30357999
XAJ347	SAMN43323394	PRJNA1151720	SRP528351	SRR30357997
XAJ349	SAMN43323395	PRJNA1151720	SRP528351	SRR30357996
XAJ35	SAMN43323396	PRJNA1151720	SRP528351	SRR30357995
XAJ355	SAMN43323397	PRJNA1151720	SRP528351	SRR30357998
XAJ36	SAMN43323398	PRJNA1151720	SRP528351	SRR30357994
XAJ372	SAMN43323399	PRJNA1151720	SRP528351	SRR30357993
XAJ38	SAMN43323400	PRJNA1151720	SRP528351	SRR30357992
XAJ390	SAMN43323401	PRJNA1151720	SRP528351	SRR30357991
XAJ402	SAMN43323402	PRJNA1151720	SRP528351	SRR30357989
XAJ426	SAMN43323403	PRJNA1151720	SRP528351	SRR30357987
XAJ427	SAMN43323404	PRJNA1151720	SRP528351	SRR30357990
XAJ428	SAMN43323405	PRJNA1151720	SRP528351	SRR30357986
XAJ432	SAMN43323406	PRJNA1151720	SRP528351	SRR30357985
XAJ435	SAMN43323407	PRJNA1151720	SRP528351	SRR30357984
XAJ438	SAMN43323408	PRJNA1151720	SRP528351	SRR30357983
XAJ439	SAMN43323409	PRJNA1151720	SRP528351	SRR30357981
XAJ456	SAMN43323410	PRJNA1151720	SRP528351	SRR30358077
XAJ458	SAMN43323411	PRJNA1151720	SRP528351	SRR30358076
XAJ462	SAMN43323412	PRJNA1151720	SRP528351	SRR30357982
XAJ464	SAMN43323413	PRJNA1151720	SRP528351	SRR30358074
XAJ465	SAMN43323414	PRJNA1151720	SRP528351	SRR30358073
XAJ466	SAMN43323415	PRJNA1151720	SRP528351	SRR30358072
XAJ467	SAMN43323416	PRJNA1151720	SRP528351	SRR30358070
XAJ469	SAMN43323417	PRJNA1151720	SRP528351	SRR30358069
XAJ470	SAMN43323418	PRJNA1151720	SRP528351	SRR30358068
XAJ473	SAMN43323419	PRJNA1151720	SRP528351	SRR30358071
XAJ476	SAMN43323420	PRJNA1151720	SRP528351	SRR30358067
XAJ477	SAMN43323421	PRJNA1151720	SRP528351	SRR30358066
XAJ669	SAMN43323422	PRJNA1151720	SRP528351	SRR30358064
XAJ670	SAMN43323423	PRJNA1151720	SRP528351	SRR30358097
XAJ671	SAMN43323424	PRJNA1151720	SRP528351	SRR30358096
XAJ672	SAMN43323425	PRJNA1151720	SRP528351	SRR30358095

Strain	BioSample	BioProject	Study	SRA Accession
XAJ673	SAMN43323426	PRJNA1151720	SRP528351	SRR30358094
XAJ674	SAMN43323427	PRJNA1151720	SRP528351	SRR30358093
XAJ675	SAMN43323428	PRJNA1151720	SRP528351	SRR30358092
XAJ729	SAMN43323429	PRJNA1151720	SRP528351	SRR30358090
XAJ730	SAMN43323430	PRJNA1151720	SRP528351	SRR30358089
XAJ731	SAMN43323431	PRJNA1151720	SRP528351	SRR30358088
XAJ929	SAMN43323432	PRJNA1151720	SRP528351	SRR30358091
XAJ930	SAMN43323433	PRJNA1151720	SRP528351	SRR30358086
XAJ934	SAMN43323434	PRJNA1151720	SRP528351	SRR30358085
XAJ935	SAMN43323435	PRJNA1151720	SRP528351	SRR30358084
XAJ936	SAMN43323436	PRJNA1151720	SRP528351	SRR30358082
XAJ939	SAMN43323437	PRJNA1151720	SRP528351	SRR30358081
XAJ940	SAMN43323438	PRJNA1151720	SRP528351	SRR30358080
XAJ968	SAMN43323439	PRJNA1151720	SRP528351	SRR30358083
XAJ969	SAMN43323440	PRJNA1151720	SRP528351	SRR30358079
XAJ973	SAMN43323441	PRJNA1151720	SRP528351	SRR30358078
XAJ984	SAMN43323442	PRJNA1151720	SRP528351	SRR30358065
XAP1007	SAMN43323443	PRJNA1151720	SRP528351	SRR30358062
XAP1010	SAMN43323444	PRJNA1151720	SRP528351	SRR30358061
XAP1159	SAMN43323445	PRJNA1151720	SRP528351	SRR30358060
XAP1265	SAMN43323446	PRJNA1151720	SRP528351	SRR30358059
XAP1657	SAMN43323447	PRJNA1151720	SRP528351	SRR30358058
XAP1741	SAMN43323448	PRJNA1151720	SRP528351	SRR30358057
XAP2004	SAMN43323449	PRJNA1151720	SRP528351	SRR30358056
XAP2016	SAMN43323450	PRJNA1151720	SRP528351	SRR30358055
XAP2023	SAMN43323451	PRJNA1151720	SRP528351	SRR30358054
XAP2029	SAMN43323452	PRJNA1151720	SRP528351	SRR30358053
XAP2036	SAMN43323453	PRJNA1151720	SRP528351	SRR30358051
XAP2039	SAMN43323454	PRJNA1151720	SRP528351	SRR30358050
XAP2106	SAMN43323455	PRJNA1151720	SRP528351	SRR30358049
XAP2113	SAMN43323456	PRJNA1151720	SRP528351	SRR30358048
XAP2150	SAMN43323457	PRJNA1151720	SRP528351	SRR30358047
XAP664	SAMN43323458	PRJNA1151720	SRP528351	SRR30358046
XAP667	SAMN43323459	PRJNA1151720	SRP528351	SRR30358045
XAP880	SAMN43323460	PRJNA1151720	SRP528351	SRR30358023
XAP883	SAMN43323461	PRJNA1151720	SRP528351	SRR30358022
XAP948	SAMN43323462	PRJNA1151720	SRP528351	SRR30358020

3.2.6 *Xaj*, *Xap*, and *Xhg* RefSeq Genomic Resources

I downloaded several RefSeq accessions for various *Xaj*, *Xap* and *X. hortorum* pv. *gardneri* (*Xhg*) for my phylogenetic analyses (3.2).

Table 3.2: NCBI RefSeq accessions for *Xap* (n=22), *Xaj* (n=26) and *Xhg* (n=1) genome(s) used in analyses. The sample collection date, host and location information for NCPPB 1447 was found in Table 1. (Scortichini et al. (2001)).

Accession	BioProject Accession	Protein Coding Genes	Gene count total	Assembly level	Organism	Number of Component Sequences	Strain	Collection Date	Host	Location
GCF_0[illegible]	P[illegible]	[illegible]	4517	Contig	Xaj	371	NCPPB 1447	1963	Juglans	Romnia
GCF_0[illegible]	P[illegible]	[illegible]	4387	Scaffold	Xaj	8	CFBP 2528	1956	Juglans regia	New Zealand
GCF_0[illegible]	P[illegible]	[illegible]	4477	Scaffold	Xaj	14	CFBP 7179	2002	Juglans regia	France: Lot-et-Garonne
GCF_0[illegible]	P[illegible]	[illegible]	4454	Chromosome	Xaj	1	Xaj 417	2012	Juglans regia	USA
GCF_0[illegible]	P[illegible]	[illegible]	4385	Contig	Xaj	161	J303	2011	Juglans regia	Chile: Concepcion
GCF_0[illegible]	P[illegible]	[illegible]	4415	Contig	Xaj	8	DW3F3	2015	walnut (Juglans regia L. cv. Qingxiang).	China: Hubei Province, Danjiangkou City
GCF_0[illegible]	P[illegible]	[illegible]	4279	Contig	Xaj	91	CFBP 8253	2002	Juglans regia cv. Franquette	France: Isere, Saint Hilaire Le Rosier

Accession	BioProject Accession	Protein Coding Genes	Gene count total	Assembly level	Organism	Number of Component Sequences	Strain	Collection Date	Host	Location
GCF_0[illegible]	P[illegible]	[illegible]	4502	Complete Genome	Xaj	1	IVIA 1317	1993	Juglans regia	Spain
GCF_0[illegible]	P[illegible]	[illegible]	4508	Complete Genome	Xaj	1	IVIA 2499	2001	Juglans regia	Spain
GCF_0[illegible]	P[illegible]	[illegible]	4235	Contig	Xaj	78	OS5	2020	Juglans	Serbia:Sirig
GCF_0[illegible]	P[illegible]	[illegible]	4471	Contig	Xaj	80	OVM2	2020	Juglans	Serbia:Milicinica
GCF_0[illegible]	P[illegible]	[illegible]	4466	Contig	Xaj	81	OVM3	2020	Juglans	Serbia:Milicinica
GCF_0[illegible]	P[illegible]	[illegible]	4248	Contig	Xaj	81	OS2	2020	Juglans	Serbia:Sirig
GCF_0[illegible]	P[illegible]	[illegible]	4473	Contig	Xaj	79	OVM1	2020	Juglans	Serbia:Milicinica
GCF_0[illegible]	P[illegible]	[illegible]	4465	Contig	Xaj	76	OC4	2020	Juglans	Serbia:Crvenka
GCF_0[illegible]	P[illegible]	[illegible]	4471	Contig	Xaj	75	OC3	2020	Juglans	Serbia:Crvenka
GCF_0[illegible]	P[illegible]	[illegible]	4468	Contig	Xaj	75	OC1	2020	Juglans	Serbia:Crvenka
GCF_0[illegible]	P[illegible]	[illegible]	4471	Contig	Xaj	75	OC2	2020	Juglans	Serbia:Crvenka
GCF_0[illegible]	P[illegible]	[illegible]	4335	Contig	Xaj	75	XjL1	2019	Juglans	Serbia:Prokuplje
GCF_0[illegible]	P[illegible]	[illegible]	4318	Contig	Xaj	82	OS3	2020	Juglans	Serbia:Sirig
GCF_0[illegible]	P[illegible]	[illegible]	4252	Contig	Xaj	77	OS4	2020	Juglans	Serbia:Sirig
GCF_0[illegible]	P[illegible]	[illegible]	4417	Scaffold	Xaj	68	WHRI 5692A	1996	Juglans regia	United Kingdom
GCF_9[illegible]	P[illegible]	[illegible]	4528	Contig	Xaj	56	CPBF 1521	2014	Juglans regia	Portugal
GCF_9[illegible]	P[illegible]	[illegible]	4490	Complete Genome	Xaj	1	CPBF 427	2016	Juglans regia	Portugal
GCF_9[illegible]	P[illegible]	[illegible]	4314	Complete Genome	Xaj	1	CPBF 1494	2014	Carya illinoinensis	Portugal
GCF_9[illegible]	P[illegible]	[illegible]	4399	Complete Genome	Xaj	1	CPBF 765	2016	Carya illinoinensis	Portugal

Accession	BioProject Accession	Protein Coding Genes	Gene count total	Assembly level	Organism	Number of Component Sequences	Strain	Collection Date	Host	Location
GCF_0[illegible]	P[illegible]	[illegible]	4353	Scaffold	Xap	77	CFBP 3894	1953	Prunus salicina	New Zealand: Auckland
GCF_0[illegible]	P[illegible]	[illegible]	4411	Contig	Xap	57	CITA 9	2008	Prunus persica	Spain: Aragon
GCF_0[illegible]	P[illegible]	[illegible]	4410	Contig	Xap	60	CITA 99	2006	Prunus dulcis	Spain: Alicante
GCF_0[illegible]	P[illegible]	[illegible]	4424	Complete Genome	Xap	1	15-088	2015	Prunus persica	South Korea
GCF_0[illegible]	P[illegible]	[illegible]	4462	Complete Genome	Xap	3	R1	2020	Prunus persica	USA: South Carolina
GCF_0[illegible]	P[illegible]	[illegible]	4604	Contig	Xap	190	F1	2020	Prunus persica	USA: South Carolina
GCF_0[illegible]	P[illegible]	[illegible]	4425	Contig	Xap	174	M1	2017	Prunus persica	USA: South Carolina
GCF_0[illegible]	P[illegible]	[illegible]	4463	Complete Genome	Xap	3	T1	2017	Prunus persica	USA: South Carolina
GCF_0[illegible]	P[illegible]	[illegible]	4441	Complete Genome	Xap	2	Xcp1	1992	Prunus persica var. nucipersica	USA:North Carolina
GCF_0[illegible]	P[illegible]	[illegible]	4426	Complete Genome	Xap	2	IVIA 2626.1	2002	Prunus salicina	Spain

Accession	BioProject Accession	Protein Coding Genes	Gene count total	Assembly level	Organism	Number of Component Sequences	Strain	Collection Date	Host	Location
GCF_02[illegible]	[illegible]	[illegible]76	4410	Complete Genome	Xap	2	CITA 33	2009	Prunus dulcis	Spain
GCF_03[illegible]	[illegible]	[illegible]36	4358	Contig	Xap	90	Xp10	2019	Prunus persica	Montenegro
GCF_03[illegible]	[illegible]	[illegible]36	4369	Contig	Xap	76	Xp720	2020	Prunus armeniaca	Serbia:Besenovo
GCF_03[illegible]	[illegible]	[illegible]36	4367	Contig	Xap	75	Xp520	2020	Prunus armeniaca	Serbia:Besenovo
GCF_03[illegible]	[illegible]	[illegible]36	4351	Contig	Xap	78	Xp419	2019	Prunus persica	Serbia:Irig
GCF_03[illegible]	[illegible]	[illegible]36	4423	Contig	Xap	79	Xp619	2019	Prunus persica	Serbia:Irig
GCF_03[illegible]	[illegible]	[illegible]36	4369	Contig	Xap	77	Xp320	2020	Prunus armeniaca	Serbia:Besenovo
GCF_03[illegible]	[illegible]	[illegible]36	4370	Contig	Xap	80	Xp1	2019	Prunus persica	Montenegro
GCF_03[illegible]	[illegible]	[illegible]36	4358	Contig	Xap	87	Xp4	2019	Prunus persica	Montenegro
GCF_03[illegible]	[illegible]	[illegible]36	4337	Contig	Xap	78	Xp7	2019	Prunus persica	Montenegro

Accession	BioProject Accession	Protein Coding Genes	Gene count total	Assembly level	Organism	Number of Component Sequences	Strain	Collection Date	Host	Location
GCF_0[illegible]	P[illegible]	4136	4363	Contig	Xap	79	Xp219	2019	Prunus persica	Serbia:Irig
GCF_0[illegible]	P[illegible]	4088	4370	Scaffold	Xap	60	PVCT 262.1	2020	Prunus dulcis	Italy:Sicily
GCF_0[illegible]	P[illegible]	4731	4687	Complete Genome	Xhg	3	JS749-3	1997	Solanum lycopersicum	Réunion

3.3 *In Vitro* Copper Sensistivity

I screened the 120 *Xaj* and 20 *Xap* strains for copper sensitivity using NA agar amended with copper sulfate ($CuSO_4$). We performed this in duplicates using the following concentrations: 0 (control), 1k, 5k, 7.5k, and 10k ppm. Strains were incubated at room temperature (22-25°C) for 3 days before measuring.

3.4 Pangenome Analyses

`PPanGGOLiN v2.2.0` (Gautreau et al. (2020)) will be used to generate the pangenomes for this study. I will run several different combinations of samples that include the RefSeq genomes (3.2). I chose to use PPanGGOLiN because I found the output files to be very user-friendly and clearly defines regions of genomic plasticity (RGPs) (e.g. Genomic Islands) and it's been widely cited (~180). The primary use of PPanGGOLiN in our study will be to generate a presence-absence matrix that will help with the identification of copper resistance-related genes, type 3 secretion system effectors, and identify RGPs associated with these respective genes.

Several runs of PPanGGOLiN were performed including:

1. California *Xaj* (n=120 and n=115)
2. California *Xap* (n=20)
3. California *Xaj* + 26 NCBI RefSeq *Xaj* strians (n=146 and n=141)
4. California *Xaj* + 24 NCBI RefSeq *Xaj* strians (No pecan strains: CPBF 1494 and CPBF 765) (n=144 and n=139)

5. California *Xap* + 22 NCBI RefSeq *Xaj* strians (n=42)
6. California *Xaj* + NCBI RefSeq *Xaj* strians + California *Xap* + NCBI RefSeq *Xaj* strians (n=188 and n=183)
7. California *Xaj* + NCBI RefSeq *Xaj* strians + California *Xap* + NCBI RefSeq *Xaj* strians + NCBI RefSeq *Xhg* (outgroup) (n=189 and n=184)
8. California *Xaj* + California *Xap* (n=140 and n=135)

NOTE: The alternative values (e.g. 115, 141, etc.) are referring to the five 'problematic strains' that were removed because the genomic data (e.g. presence of copper resistance-related genes) did not align with the *in vitro* copper sensitivity data (See Section 4.3).

Here are the respective directories for the analyses above (Not shown are the directories with _115):

1. CA_XAJ
2. CA_XAP
3. CA_XAJ_and_NCBI26
4. CA_XAJ_and_NCBI24
5. CA_XAP_and_NCBI22
6. CA_XAJ_XAP_and_NCBI_XAJ_XAP
7. CA_XAJ_XAP_and_NCBI_XAJ_XAP_and_Outgroup

PPanGGOLiN Workflow

1. Run PPanGGOLiN using the `-all` mode to run all the pangenome analyses using PPanggolin.sh (5.3.2).
2. Generate MSA of the 'shell' genome for each run. (NEED TO RUN 11/25/24)

PPanGGOLiN requires a `.tsv` file that contains two columns: strain and path to `.gbk` files. I created a `.tsv` named after the respective directories above but with the `_Path.tsv` suffix. Each run had its own script, and both the script and .tsv files were in their respective directories, see PPanggolin_TSV.sh (5.3.1).

HPCC working directory:

`/bigdata/adaskaveglab/ashan007/Assembly_Workflow_PlatanusB/Step5_Analyses/Pangenome/Ppa`

Local working directory:

`/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5_Analyses/Pangenome/P`

3.5 Phylogenetic Analyses

Phylogenetic analysis was conducted using the workflow described in Figure 3.2.

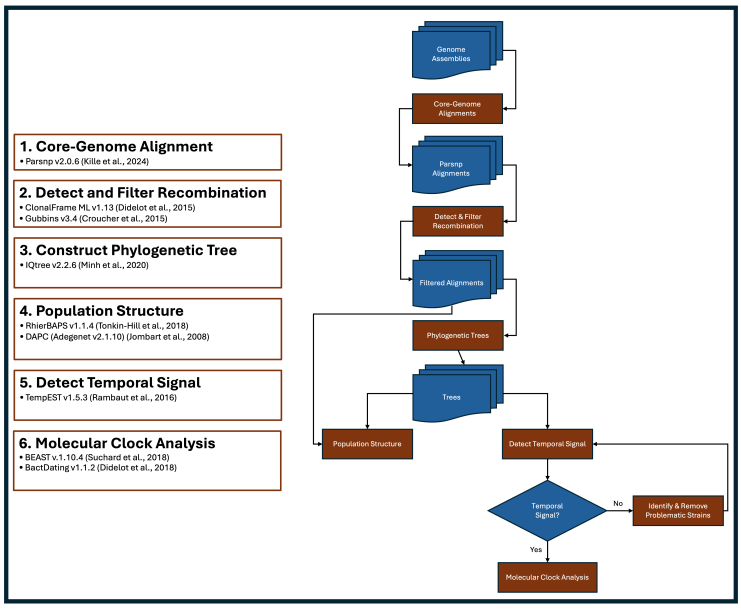


Figure 3.2: Phylogenetic workflow

3.5.1 Core-Genome Alignments

I downloaded `Parsnp v2.0.6` (Kille et al. (2024)) to perform whole-genome core alignments. I used IVIA 1317 as a reference for *XaJ* and I used isolate 15-088 as the *Xap* reference therefore, I did not need to supply it as one of the genomes (in the /bigdata/adaskaveglab/ashan007/Assembly_Workflow_PlatanusB/Step5_Analyses/Genomic_Resources/Genome directory) since it is included in the trees. Other than the CA_XAP and CA_XAP_and_NCBI22 runs, the reference is IVIA 1317. I then used `HarvestTools v1.3` (Treangen et al. (2014)) to convert the `.xmfa` to `.aln` (fasta) and concatenate the Local Colinear Blocks (LCBs).

- I removed the IVIA 1317 genomes from: CA_XAJ_and_NCBI26, CA_XAJ_XAP_and_NCBI_XAJ_XAP and CA_XAJ_XAP_and_NCBI_XAJ_XAP_and_Outgroup Genomes subdirectories.
- I removed the 15-088 genome from: CA_XAP_and_NCBI22 Genomes subdirectory.
- I also modified the `.aln` files to remove `.fa`, `.fna`, `.gbk`, or `.gbk.fna` extension in the sample name using `awk`. This code was applied to all runs:

```
awk '/^>/ { sub(/\.gbk\.fna$/, ""); sub(/\.gbk$/, ""); sub(/\.fna$/, ""); sub(/\.fa$/, "");
```

- I then checked it see if the sample names were correctly renamed using `grep` + `sed`:

```
grep '^>' CA_XAJ.LCBs.fasta | sed 's/^>//' > CA_XAJ.LCBs.names.txt
```

Scripts

I used `Parsnp_Final.sh` (5.4.1) to generate the initial alignments and `HarvestTools_Final.sh` (5.4.2) to extract the LCBs.

3.5.2 Detect and Filter Recombination

`Gubbins v3.4` (Croucher et al. (2015)) and `ClonalFrameML v1.13` (Didelot and Wilson (2015)) (CFML) are two commonly used programs to detect and filter bacterial recombination. In order to produce accurate phylogenies we must account for recombination. I will use these two methods and compare the results (e.g. core-genome alignments) using `AMAS` (Borowiec (2016)).

Relevant Literature Phylogentetic Analyses

- Parsnp + ClonalFrameML
  - Emergence of linezolid-resistant *Enterococcus faecium* in a tertiary hospital in Copenhagen (2023) (https://doi.org/10.1099/mgen.0.001055)
  - Emergence and spread of *Mycobacterium ulcerans* at different geographic scales (2024) (https://journals.asm.org/doi/full/10.1128/spectrum.03827-23)
  - Whole genome sequencing for investigations of meningococcal outbreaks in the United States: a retrospective analysis (2018) (https://doi.org/10.1038/s41598-018-33622-5)
  - Acquisition and dissemination of cephalosporin-resistant *E. coli* in migratory birds sampled at an Alaska landfill as inferred through genomic analysis (2018) (https://doi.org/10.1038/s41598-018-25474-w)
  - The Green Revolution shaped the population structure of the rice pathogen *Xanthomonas oryzae* pv. *oryzae* (2020) (https://doi.org/10.1038/s41396-019-0545-2)
  - Evolutionary and epidemiological insights from historical and modern genomes of *Xanthomonas oryzae* pv. *oryzicola*, the causal agent of bacterial leaf streak of rice (2024) (https://doi.org/10.1094/MPMI-05-24-0062-SC)
- Parsnp + Gubbins
  - Genomic Traits Associated with Virulence and Antimicrobial Resistance of Invasive Group B Streptococcus Isolates with Reduced Penicillin Susceptibility from Elderly Adults (2022) (https://doi.org/10.1128/spectrum.00568-22)
  - Population genomics and pathotypic evaluation of the bacterial leaf blight pathogen of rice reveals rapid evolutionary dynamics of a plant pathogen (2023) (https://doi.org/10.3389/fcimb.2023.1183416)

- Natural Recombination among Type I Restriction-Modification Systems Creates Diverse Genomic Methylation Patterns among *Xylella fastidiosa* Strains (2023) (https://doi.org/10.1128/aem.01873-22)
- Genomic analysis of the multi-host pathogen *Erysipelothrix rhusiopathiae* reveals extensive recombination as well as the existence of three generalist clades with wide geographic distribution (2016) (https://doi.org/10.1186/s12864-016-2643-0)
- Global phylogenomic analyses of *Mycobacterium abscessus* provide context for non cystic fibrosis infections and the evolution of antibiotic resistance (2021) (https://doi.org/10.1038/s41467-021-25484-9)

3.5.3 Phylogenetic Tree

`IQtree2 v.2.2.6` (Minh et al. (2020)) will be used to generate the phylogenetic tree based on the recombination-filtered genome alignments from `CFML` and `Gubbins`. Prior to generating the tree I ran `ModelFinder` (Kalyaanamoorthy et al. (2017)) to determine the optimal substitution model based on the alignments (Table 3.3). I compared the trees generated by `IQtree` + `Gubbins` and `IQtree` + `ClonalFrameML` using TreeCMP (Goluch et al. (2020)).

Models determined by ModelFinder

The models are found within the `MFout.iqtree` file in the respective Parsnp run subdirectories, for example:

`/bigdata/adaskaveglab/ashan007/Assembly_Workflow_PlatanusB/Step5_Analyses/IQtree2/Parsnp_CFML/CA_`

`/bigdata/adaskaveglab/ashan007/Assembly_Workflow_PlatanusB/Step5_Analyses/IQtree2/Parsnp_Gubbins/`

Table 3.3: Models determined by `ModelFinder`.

Run	ClonalFrameML	Gubbins	Outgroup
CA_XAJ	HKY+F+I	HKY+F+I	NONE
CA_XAJ_and_NCBI26	TPM3u+F+I	TPM3u+F+I	NONE
CA_XAJ_XAP_and_NCBI24	GTR+F+I	GTR+F+I	NONE
CA_XAJ_XAP_and_TWCB1	GTR+F+I	GTR+F+I	JS749_3
CA_XAJ_XAP	HKY+F+I	GTR+F+I	NONE
CA_XAJ_and_NCBI24	HKY+F+I	HKY+F+I	NONE
CA_XAP	K3Pu+F+I	K3Pu+F+I	NONE

Scripts ClonalFrameML: ModelFinder and IQtree were ran using `IQTree2_MF_Parsnp_CFML_Final.sh` (5.4.3) and `IQTree2_Parsnp_CFML_Final.sh` (5.4.4), respectively.
Gubbins: ModelFinder and IQtree were ran using `IQTree2_MF_Parsnp_Gubbins_Final.sh` (5.4.5) and `IQTree2_Gubbins_Final.sh` (5.4.6), respectively.

3.5.4 Population Structure

To complement and support the phylogenies, I wanted to elucidate the population structure using two methods: `RhierBAPS` (Tonkin-Hill et al. (2018)) and DAPC (Jombart (2008)). `RhierBAPS` is a Bayesian clustering approach and DAPC is a model-free approach that combined PCA (Principal Component Analysis) for dimensionality reduction and DA (Discriminant Analysis) to maximize the separation between pre-defined or inferred clusters.

First I ran RheirBAPS to generate the clusters based on the input recombination-filtered genome alignment and respective IQtree tree. The recombination-filtered genome alignment was converted into a SNP matrix then into a `genind` object in order to perform the DAPC. The clusters then served as the groups (`grp`) for the DAPC analysis.

Scripts:

I will only provide the scripts that we will likely use for the manuscripts.

Parsnp + CFML + IQtree

- CA_XAJ: `CA_XAJ_hierBAPS.R`
- CA_XAJ_115: `CA_XAJ_hierBAPS.R` ((5.4.7))
- CA_XAJ_115_and_NCBI24: `CA_XAJ_and_NCBI24_hierBAPS.R` ((5.4.8))
- CA_XAJ_115_and_NCBI26: `CA_XAJ_and_NCBI26_hierBAPS.R`
- CA_XAJ_115_XAP: `CA_XAJ_hierBAPS.R`
- CA_XAJ_115_XAP_and_NCBI_XAJ_XAP: `CA_XAJ_XAP_and_NCBI_XAJ_XAP_hierBAPS.R`
- CA_XAJ_115_XAP_and_NCBI_XAJ_XAP_and_Outgroup: `CA_XAJ_XAP_and_NCBI_XAJ_XAP_and_Outgroup_hierBAPS.R` ((5.4.9))
- CA_XAJ_and_NCBI24: `CA_XAJ_and_NCBI24_hierBAPS.R`
- CA_XAJ_and_NCBI26: `CA_XAJ_and_NCBI26_hierBAPS.R`
- CA_XAJ_XAP: `CA_XAJ_XAP_hierBAPS.R`
- CA_XAJ_XAP_and_NCBI_XAJ_XAP: `CA_XAJ_XAP_and_NCBI_XAJ_XAP_hierBAPS.R`
- CA_XAJ_XAP_and_NCBI_XAJ_XAP_and_Outgroup: `CA_XAJ_XAP_and_NCBI_XAJ_XAP_and_`
- CA_XAP: `CA_XAP_hierBAPS.R`
- CA_XAP_and_NCBI22: `CA_XAP_NCBI22_hierBAPS.R`

3.5.5 Molecular Clock

I wanted to see if our data was 'clock-like' and fit for a dated phylogenetic analysis. To determine if the data is 'clock-like' you must detect a significant temporal signal by performing a root-to-tip regression analysis.

"A data set is considered to have temporal signal if it can be treated as a measurably evolving population, defined by Drummond et al. (2003) as "populations from which molecular sequences can be taken at different points in time, among which there are a statistically significant number of genetic differences. (Duchene et al. (2020))"

If there is a weak temporal signal, then the data does not exhibit clock-like behavior and is unfit for molecular dating analysis.

I used two programs to detect temporal signal:

- `TempEST v1.5.3` (Rambaut et al. (2016))
  - TempEst (short for "Temporal Explorer of Sequence Trees," formerly known as Path-O-Gen) is a tool designed for analyzing the temporal signal and "clocklikeness" of molecular phylogenies. It's widely used in evolutionary biology to assess whether a dataset of genetic sequences (e.g., from viruses or bacteria) shows a measurable rate of evolution over time. TempEst can handle both contemporaneous trees (all sequences collected at the same time) and dated-tip trees (sequences collected at different times). It helps researchers visualize the relationship between genetic divergence and sampling dates through root-to-tip regression plots, and it can suggest an optimal root for the tree that aligns with a molecular clock assumption.
  - Since `TempEST` is a stand-alone software package with a GUI, I do not have any associated code. I followed the tutorial described here.
- `BactDating v1.1.2` (Didelot et al. (2018))
  - BactDating is a Bayesian tool for dating ancestral nodes in bacterial phylogenetic trees, integrating molecular clock models with epidemiological or historical data. It's particularly useful for reconstructing the evolutionary history of bacterial pathogens, estimating when key events (like outbreaks or divergences) occurred. Built in R, it uses a relaxed molecular clock and can incorporate uncertainty in tip dates (sampling times), making it flexible for messy real-world datasets.

I performed temporal analysis using `BactDating` on the CA_XAJ and CA_XAJ_and_NCBI24 runs using the following data and scripts:

- ClonalFrameML output
  - CA_XAJ script: `CA_XAJ_CFML_BactDating.R` (5.5.1)
  - CA_XAJ_and_NCBI24 script: `CA_XAJ_NCBI24_CFML_BactDating.R`
- Gubbins output
  - CA_XAJ script: `CA_XAJ_Gub_BactDating.R` (5.5.2)
  - CA_XAJ_and_NCBI24 script: `CA_XAJ_NCBI24_Gub_BactDating.R`
- IQtree2 tree based on the CFML-filtered alignment
  - CA_XAJ script: `CA_XAJ_CFML_IQtree_BactDating.R` (5.5.3)
  - CA_XAJ_and_NCBI24 script: `CA_XAJ_NCBI24_CFML_IQtree_BactDating.R`
- IQtree2 tree based on the Gubbins-filtered alignment
  - CA_XAJ script: `CA_XAJ_Gub_IQtree_BactDating.R` (5.5.4)
  - CA_XAJ_and_NCBI24 script: `CA_XAJ_NCBI24_Gub_IQtree_BactDating.R`

NOTE: I only provided one example for each of the data because the code is the same for the runs.

3.6 Type III Effectors

I downloaded Xanthomonas type III secretion system effectors (T3E) (n= 76) (Accessed July 19, 2024) from Euroxanth to serve as BLAST reference/queries for BlastP and tBLASTn against our CA Xaj and Xap populations. See `Xanthomonas_Effectors.xlsx` for more information on these reference T3Es and the preliminary results.

This work was performed on the HPCC in the following directory:

`/bigdata/adaskaveglab/ashan007/Assembly_Workflow_PlatanusB/Step5_Analyses/Effectors`

BLAST

- tBLASTn was ran using `Xaj_Xap_T3SSE_tBLASTn.sh` (5.6.1)
- BLASTp was ran using `Xaj_Xap_T3SSE_BLASTp.sh` (5.6.2)
- Both BLASTp and tBLASTn had these settings:

```
-evalue 1e-10
-num_threads 8
-max_target_seqs 5
-max_hsps 5
```

- Both the .txt and alignment outputs were generated.

3.6.0.1 Method 1

Subedi et al. (2024)

"The presence of 72 T3Es (Supplementary Table S2) was investigated for all genomes using tBLASTn with E-value greater than 10–5. A threshold of > 80% amino acid identity covering >80% of query length was considered as the presence of an effector." with the addition of BLASTp using the same filtering criteria.

Steps:

1. Run tBLASTn using `Xaj_Xap_T3SSE_tBLASTn.sh` (5.6.1).
2. Filter the tBLASTn output for effectors using a threshold of > 80% amino acid identity covering > 80% of query length using `Filter_tBLASTn.py` (5.6.3).
3. Identify duplicate hits for qseqid and remove the hits with a lower percent ID (my modification) using `Filter_tBLASTn.RemoveDups.py` (5.6.4).
4. Generate a presence/absence matrix using `BLAST_Matrix_FamilyName.method1.py` (5.6.5) and `BLAST_Matrix_RefSeqID.method1.py` (5.6.6).

3.6.0.2 Method 2

Fernandes et al. (2021)

"The genome sequences of the four Xanthomonas strains used in this study were scrutinized for the presence of protein homologs by tBLASTn analysis (tBLASTn v. 2.10.1) against a created database of protein sequences previously reported to be involved in the pathogenesis and virulence of Xanthomonas (Table S3). To ensure that only closely-related orthologous proteins were selected, the tBLASTn cut-offs criteria to identify protein homologs were $\geq$ 40% identity and $\geq$ 75% query sequence length."

Steps:

1. Run tBLASTn using `Xaj_Xap_T3SSE_tBLASTn.sh` (5.6.1).
2. Filter the tBLASTn output for effectors using a threshold of $\geq$ 40% identity and $\geq$ 75% query sequence length using `Filter_tBLASTn.v2.py` (5.6.7).
3. Identify duplicate hits for qseqid and remove the hits with a lower percent ID (my modification) using `Filter_tBLASTn.RemoveDups.py` (5.6.4).
4. Generate a presence/absence matrix using `BLAST_Matrix_FamilyName.method2.py` (5.6.8) and `BLAST_Matrix_RefSeqID.method2.py` (5.6.9).

3.6.0.3 Method 3

Assis et al. (2021)

"A set of 63 type III effectors from the Xanthomonas Resource website was analyzed by protein-nucleotide six-frame translation (tBlastn) searches [74] with E-value 1e-50 against the set of 32 Xaj genomes."

Steps:

1. Run tBLASTn using `Xaj_Xap_T3SSE_tBLASTn.sh` (5.6.1).
2. Filter the tBLASTn output for effectors using a threshold of E-value $\leq 1e-50$ using `Filter_tBLASTn.v3.py` (5.6.10).
3. Generate a presence/absence matrix using `BLAST_Matrix_FamilyName.v2.py` (5.6.11) and `BLAST_Matrix_RefSeqID.v2.py` (5.6.12).

3.6.0.4 Method 4

Inspecting the PPanGGoLiN data matrix for the presence of annotated T3Es.

- (CA_XAJ_XAP run: CA_XAJ_XAP_PPanGGOLiN_Matrix.xlsx)
- (CA_XAJ_115 run: matrix_Xaj_115.xlsx)

Steps

1. CA_XAJ_XAP_PPanGGOLiN_Matrix.xlsx was used to identify effectors by name and then by searching for genes present in method 3 by gene ID to identify those genes in the PPanGGOLiN results.
2. for select genes I took a representative sequence from one strain and performed a web-based BLASTp to confirm that the protein is the respective effector. See Note 4 on the Preliminary_Results tab of `Xanthomonas_Effectors.xlsx`.

Chapter 4

Results

4.1 Read Processing

On average, 3.92 ± 0.44 G of raw data was generated for each strain (Table 4.1).

Table 4.1: Sequencing results from Novogene.

Sample	Raw reads	Raw data	Effective(%)	Error(%)	Q20(%)	Q30(%)	GC(%)
XAJ1028	25591934	3.8	99.8	0.03	97.84	94.32	65.61
XAJ1046	27128744	4.1	99.78	0.03	97.84	94.33	65.62
XAJ1048	22783072	3.4	99.78	0.02	97.91	94.49	65.53
XAJ1049	24548562	3.7	99.72	0.02	97.91	94.51	65.47
XAJ1054	21141570	3.2	99.4	0.03	97	93.14	65.35
XAJ1082	23517808	3.5	99.79	0.02	97.92	94.52	65.58
XAJ1086	22371990	3.4	99.79	0.02	97.96	94.58	65.6
XAJ1148	25554480	3.8	99.79	0.02	97.96	94.6	65.53
XAJ1149	24621852	3.7	99.81	0.02	97.86	94.42	65.59
XAJ1197	26704772	4	99.82	0.03	97.79	94.26	65.48
XAJ1213	22911558	3.4	99.83	0.02	97.88	94.38	65.46
XAJ1239	28284048	4.2	99.82	0.03	97.77	94.2	65.54
XAJ1246	24498244	3.7	99.82	0.02	97.92	94.46	65.49
XAJ1250	27743394	4.2	99.85	0.02	97.92	94.52	65.48
XAJ1395	23141436	3.5	99.78	0.02	98.2	95.06	65.49
XAJ1399	23736590	3.6	99.7	0.02	98.12	94.94	65.49
XAJ145	30305706	4.5	99.79	0.03	97.7	93.98	65.55
XAJ1498	22844534	3.4	99.73	0.02	98.11	94.9	65.54
XAJ1535	20554072	3.1	99.73	0.02	97.95	94.38	60.1
XAJ1537	28392148	4.3	99.75	0.02	98.14	94.97	65.47
XAJ1539	25641200	3.8	99.77	0.02	98.1	94.88	65.54

Sample	Raw reads	Raw data	Effective(%)	Error(%)	Q20(%)	Q30(%)	GC(%)
XAJ1551	24648774	3.7	99.76	0.02	98.17	95.03	65.63
XAJ1554	24667592	3.7	99.81	0.02	98.12	94.93	65.64
XAJ1556	26642296	4	99.77	0.02	98.11	94.91	65.53
XAJ1557	32063060	4.8	99.74	0.02	98.18	95.03	65.48
XAJ1558	22755570	3.4	99.79	0.02	98.16	94.97	65.5
XAJ1595	26746834	4	99.8	0.03	97.8	94.17	65.51
XAJ1606	22419850	3.4	99.83	0.02	97.95	94.57	65.48
XAJ1613	27831348	4.2	99.79	0.03	97.79	94.18	65.4
XAJ1614	27277774	4.1	99.8	0.03	97.75	94.1	65.53
XAJ1615	26327478	3.9	99.8	0.02	97.86	94.34	65.34
XAJ1616	30127594	4.5	99.82	0.03	97.77	94.12	65.55
XAJ1617	27929602	4.2	99.8	0.03	97.77	94.11	65.62
XAJ1626	26635082	4	99.78	0.03	97.81	94.21	65.52
XAJ163	27523144	4.1	99.81	0.02	97.88	94.37	65.5
XAJ165	29565610	4.4	99.83	0.03	97.69	93.98	65.5
XAJ169	29377416	4.4	99.83	0.03	97.82	94.24	65.5
XAJ1710	23142910	3.5	99.81	0.03	97.74	94.07	65.41
XAJ1711	25659456	3.8	99.81	0.03	97.75	94.09	65.36
XAJ1808	26157004	3.9	99.84	0.03	97.61	93.78	65.55
XAJ181	30304682	4.5	99.83	0.03	97.75	94.07	65.49
XAJ1822	25639610	3.8	99.84	0.03	97.65	93.89	65.51
XAJ1824	27202276	4.1	99.83	0.03	97.83	94.25	65.53
XAJ2122	29431940	4.4	99.77	0.02	97.89	94.38	65.52
XAJ2124	24947322	3.7	99.85	0.02	97.87	94.33	65.32
XAJ2125	21836920	3.3	99.79	0.03	97.77	94.18	65.32
XAJ2128	27026812	4.1	99.87	0.02	97.9	94.44	65.25
XAJ2129	28275218	4.2	99.86	0.02	97.91	94.48	65.29
XAJ2131	29505070	4.4	99.82	0.03	97.81	94.27	65.46
XAJ2165	16555726	2.5	99.74	0.02	97.9	94.48	65.35
XAJ2167	27266132	4.1	99.84	0.02	97.89	94.46	65.28
XAJ2169	26778130	4	99.85	0.02	97.98	94.59	64.07
XAJ2170	23357712	3.5	99.85	0.03	97.79	94.22	65.4
XAJ2178	25383038	3.8	99.82	0.02	97.9	94.48	65.4
XAJ2180	25701552	3.9	99.79	0.02	97.86	94.37	65.58
XAJ283A	31719184	4.8	99.75	0.03	97.76	94.14	65.55
XAJ283B	23850040	3.6	99.58	0.02	97.86	94.35	65.53
XAJ30	28549684	4.3	99.83	0.03	97.85	94.28	65.68
XAJ31	30944874	4.6	99.79	0.03	97.65	93.89	65.69
XAJ32	26826246	4	99.83	0.03	97.67	93.92	65.39
XAJ321	28989620	4.3	99.83	0.03	97.73	94.05	65.36
XAJ322	29391010	4.4	99.64	0.03	97.61	93.81	65.52
XAJ33	29806428	4.5	99.56	0.03	97.28	93.22	65.51
XAJ330	25695714	3.9	99.8	0.03	97.71	94.01	65.48

Sample	Raw reads	Raw data	Effective(%)	Error(%)	Q20(%)	Q30(%)	GC(%)
XAJ333	25529262	3.8	99.74	0.03	97.83	94.28	65.46
XAJ338	29804220	4.5	99.76	0.02	97.92	94.46	65.51
XAJ34	22366134	3.4	99.85	0.03	97.74	94.08	65.42
XAJ340	30453802	4.6	99.82	0.02	97.9	94.44	65.43
XAJ341	22967944	3.4	99.76	0.03	97.72	94.21	64.7
XAJ344	32172478	4.8	99.81	0.03	97.84	94.33	65.16
XAJ345	26215790	3.9	99.81	0.03	97.85	94.33	65.5
XAJ347	23904350	3.6	99.56	0.02	98.23	95.18	65.32
XAJ349	24222738	3.6	99.5	0.02	98.21	95.12	65.46
XAJ35	30751938	4.6	99.8	0.03	97.65	93.84	65.49
XAJ355	26167538	3.9	99.76	0.03	97.81	94.24	65.4
XAJ36	25042538	3.8	99.8	0.03	97.72	94.04	65.63
XAJ372	26460434	4	99.72	0.02	98.05	94.71	65.5
XAJ38	34529828	5.2	99.81	0.03	97.69	93.95	65.47
XAJ390	28702950	4.3	99.81	0.03	97.78	94.21	65.59
XAJ402	24876514	3.7	99.77	0.02	97.93	94.53	65.48
XAJ426	25297070	3.8	99.79	0.03	97.74	94.1	65.46
XAJ427	22565974	3.4	99.81	0.02	98.05	94.72	64.6
XAJ428	24115680	3.6	99.75	0.02	98.18	95.05	65.51
XAJ432	24746174	3.7	99.78	0.02	98.17	95.03	65.52
XAJ435	22312592	3.3	99.77	0.02	98.16	94.97	65.45
XAJ438	25001438	3.8	99.78	0.02	98.19	95.05	65.51
XAJ439	23152004	3.5	99.75	0.02	98.15	94.95	65.5
XAJ456	25371674	3.8	99.78	0.02	98.26	95.21	65.56
XAJ458	24367088	3.7	99.73	0.02	98.04	94.73	65.57
XAJ462	25507154	3.8	99.77	0.02	98.11	94.9	65.52
XAJ464	25831450	3.9	99.12	0.02	97.89	94.37	65.61
XAJ465	22375006	3.4	99.67	0.02	98.09	94.85	65.6
XAJ466	24006834	3.6	99.64	0.02	98.06	94.8	65.59
XAJ467	24139960	3.6	99.68	0.02	98.16	95.03	65.59
XAJ469	25378860	3.8	99.82	0.02	97.98	94.58	65.63
XAJ470	26132060	3.9	99.76	0.02	97.92	94.5	65.62
XAJ473	27096368	4.1	99.79	0.03	97.75	94.07	65.65
XAJ476	32787634	4.9	99.74	0.02	97.94	94.52	65.63
XAJ477	27245078	4.1	99.81	0.02	97.96	94.57	65.67
XAJ669	23070940	3.5	99.8	0.02	98.09	94.78	65.47
XAJ670	26035570	3.9	99.74	0.02	97.93	94.52	65.61
XAJ671	28669172	4.3	99.74	0.02	97.97	94.57	65.56
XAJ672	24008700	3.6	99.75	0.02	97.9	94.4	65.43
XAJ673	26955266	4	99.76	0.03	97.82	94.25	65.44
XAJ674	26455670	4	99.77	0.03	97.82	94.26	65.61
XAJ675	23235694	3.5	99.78	0.02	97.87	94.37	65.63
XAJ729	26190522	3.9	99.77	0.02	98.03	94.72	65.53

Sample	Raw reads	Raw data	Effective(%)	Error(%)	Q20(%)	Q30(%)	GC(%)
XAJ730	28142470	4.2	99.77	0.02	97.91	94.5	65.41
XAJ731	35859524	5.4	99.74	0.03	97.82	94.28	65.51
XAJ929	26992496	4	99.72	0.02	97.86	94.41	65.57
XAJ930	24434222	3.7	99.74	0.02	97.98	94.59	65.54
XAJ934	26225454	3.9	99.78	0.02	97.94	94.55	65.57
XAJ935	25156038	3.8	99.72	0.02	97.86	94.41	65.58
XAJ936	21308058	3.2	99.76	0.02	97.91	94.49	65.57
XAJ939	25645362	3.8	99.81	0.03	97.8	94.23	65.42
XAJ940	27863848	4.2	99.77	0.02	97.94	94.5	65.35
XAJ968	21680944	3.3	99.8	0.02	97.87	94.41	65.64
XAJ969	22272334	3.3	99.74	0.03	97.83	94.32	65.59
XAJ973	23804496	3.6	99.56	0.03	97.76	94.16	65.49
XAJ984	21104792	3.2	98.65	0.03	96.53	92.82	65.19
XAP1007	23944430	3.6	99.8	0.02	98.05	94.72	65.71
XAP1010	27496956	4.1	99.82	0.02	98.13	94.92	65.7
XAP1159	28628180	4.3	99.78	0.02	98.19	95.05	65.14
XAP1265	28189252	4.2	99.82	0.02	98.16	94.99	65.24
XAP1657	29414488	4.4	99.84	0.03	97.77	94.15	65.72
XAP1741	29420356	4.4	99.84	0.03	97.7	93.98	65.3
XAP2004	24706482	3.7	99.83	0.03	97.75	94.08	65.72
XAP2016	28494208	4.3	99.8	0.03	97.79	94.14	65.79
XAP2023	27413818	4.1	99.85	0.03	97.72	94.04	65.73
XAP2029	25694988	3.9	99.84	0.03	97.76	94.1	65.5
XAP2036	26720266	4	99.83	0.03	97.78	94.17	65.27
XAP2039	32014862	4.8	99.82	0.03	97.85	94.31	65.65
XAP2106	26342358	4	99.83	0.02	97.85	94.37	65.68
XAP2113	30149044	4.5	99.85	0.02	97.91	94.45	65.42
XAP2150	24668520	3.7	99.86	0.02	97.9	94.45	65.4
XAP664	26119486	3.9	99.83	0.02	97.98	94.62	65.48
XAP667	25615846	3.8	99.85	0.03	97.78	94.21	65.49
XAP880	23391520	3.5	99.81	0.03	97.78	94.23	65.69
XAP883	23847148	3.6	99.83	0.02	97.85	94.36	65.68
XAP948	24923122	3.7	99.77	0.02	98.14	94.92	65.05

4.2 Genome Assembly and Annotation

The assembly pipeline was successful and generated contiguous, high-quality draft assemblies for the *Xaj* and *Xap* populations. The *Xaj* assemblies were comprised of 61.4 ± 7.4 contigs with a length of 5.14 ± 0.07 Mb and an N50 of 167.1 Kb ± 27.7 Kb. The *Xaj* PGAP annotations yielded 4,485.8 ± 79.1 genes and 4,217 ± 73 proteins. These assembly and annotation results are consistent with other *Xaj* assemblies (n=32) that were included in Table S1 described in

Assis et al. (2021). The results from CheckM run on the polished assemblies are found in Table 4.2.

The *Xap* assemblies were comprised of 34.9 ± 7 contigs with a length of 4.97 ± 0.02 Mb and an N50 317.6 ± 139.2 Kb. The *Xap* PGAP annotations yielded 4,276.7 ± 22.6 genes and 3,987 ± 24.2 proteins. These assembly and annotation reports were consistent with other publicly available Xap genomic resources (accessed August 24, 2024). The results from PGAP are found in 4.3.

All the ANI reports indicated the isolates were `CONFIRMED` with `HIGH` confidence in the assigned taxonomy with ≥ 98.081 (Xaj) and ≥ 97.299 (Xap) identity to *Xanthomonas arboricola* pv. *juglandis* (GCA_001013475.1, ASM101347v1) in `ani_summary_table.tsv`. CheckM reports indicated ≥ 97.03 and 98.62% completeness and ≤ 1.32 and 1.87% contamination for the Xaj and Xap populations, respectively.

Overall, I'm satisfied with the genome assembly and annotation results of the *Xap* and *Xaj* populations. Our results are consistent with other published assemblies and will serve as a reliable resource for future studies.

Table 4.2: Summarized *Xaj* and *Xap* CheckM (non-PGAP run) results. This data was generated by using Summarize_CheckM.sh (5.2.3)

Pathovar	Raghotama Reference	CheckM Completeness (%)	CheckM Contamination (%)	GC %	Genbase size	Number of ambiguous bases (Ns)	Number of scaffolds	Number of contigs	Longest scaffold	Longest contig	N50 scaffold	N50 contig	Scaffold length	Number of predicted genes	Genome Size Mb (No Ns)
Xaj	IVIA 1947	99.19	0.93	65	5,261,237	7,086	61.45	5,261,423	536,190	275,861	182,252	134,587	5	2.16	
Xap	15_08	98.70	0.71	66	5,090,130	6,300	64.95	5,090,694	520,963	317,164	150,038	187,198	8	2.16	

Table 4.3: *Xaj* and *Xap* annotation results. This data was generated by using Summarize_Genbank.py (5.2.4).

Strain	Genes (total)	CDSs (total)	Genes (coding)	CDSs (with protein)	Genes (RNA)	tRNAs	ncRNAs	Pseudo Genes (total)	CDSs (without protein)
Xaj	4485.2 ± 79	4383.3 ± 79.8	4217 ± 73.1	4217 ± 73.1	102.5 ± 3.9	54.7 ± 1.5	42 ± 2.9	166.3 ± 14.8	166.3 ± 14.8

Strain	Genes (total)	CDSs (total)	Genes (coding)	CDSs (with protein)	Genes (RNA)	tRNAs	ncRNAs	Pseudo Genes (total)	CDSs (without protein)
Xap	4276.7 ± 22.7	4181.4 ± 22.2	3987.5 ± 24.3	3987.5 ± 24.3	95.3 ± 0.6	53.3 ± 0.6	36 ± 0	193.9 ± 7.1	193.9 ± 7.1

4.3 *In Vitro* Copper Sensitivity

We identified 77 *Xaj* strains that were resistant to copper, the remaining *Xaj* and all *Xap* strains were sensitive. Resistance was characterized by growth at $\geq 5,000 ppm$ (4.1, 4.2, 4.3).

I identified five strains, XAJ321, XAJ341, XAJ939, XAJ1614, XAJ355, that did not align with the genomic data. These strains were all found to be resistant but did not encode copper resistance-related genes (see SECTION 4.4 Pangenome results). I removed these strains from the study and re-ran the pangenome and phylogenetic analyses. These updated runs have "XAJ_115" in the name. The results throughout this book will reflect these changes.

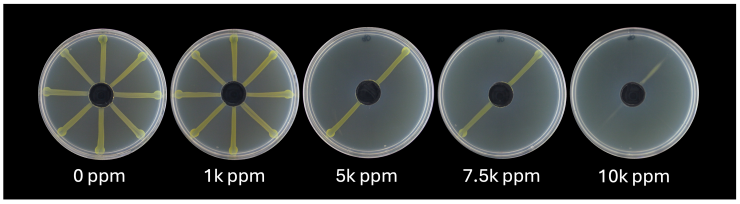


Figure 4.1: *In vitro* copper sensitivity. Pictured are strains XAJ1606, XAJ355, XAP880, and XAP883.

4.4 Pangenome Analyses (NEED TO FINISH)

The pangenome analysis using `PPanGGoLiN` was successful. I will describe the results based on the respective runs: CA_XAJ_115 (n=115) and CA_XAJ_115_XAP (n=135), with more emphasis on the CA_XAJ_115 since that will be the data we use for the manuscript. The gene presence/absence data was used to identify genes related to copper resistance as well as Type III effectors. The pangenome graph was used to visualize the various partitions ('persistent', 'shell', and 'cloud').

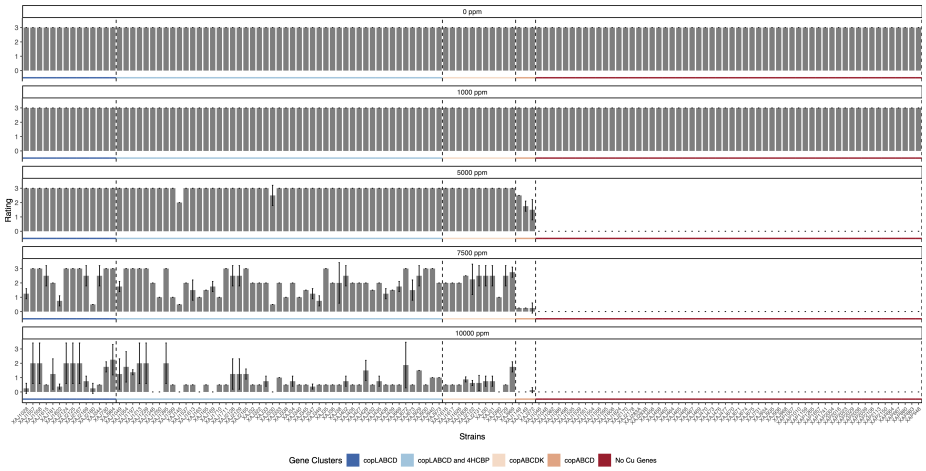


Figure 4.2: *In vitro* copper sensitivity results for all strains (n = 135).

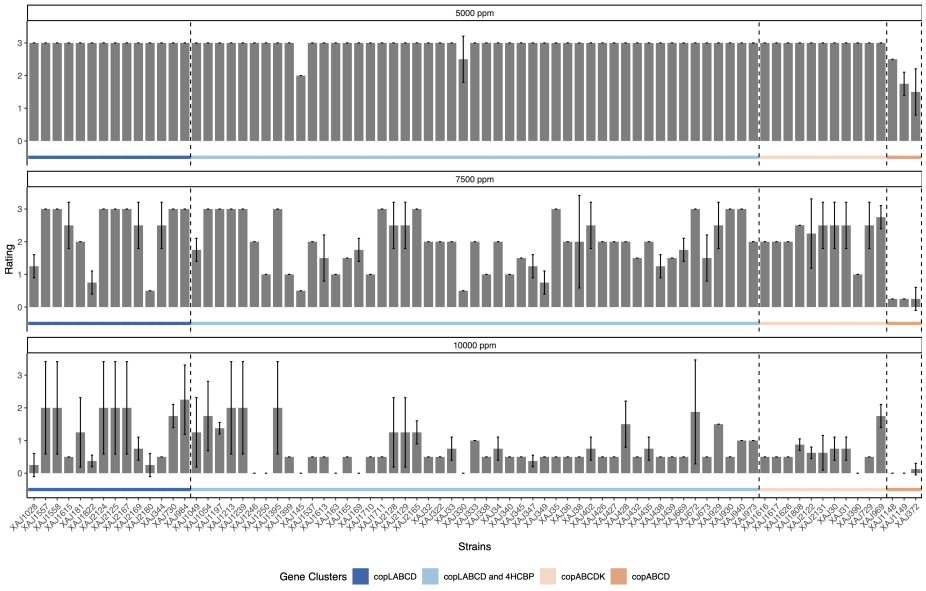


Figure 4.3: *In vitro* copper sensitivity of the copper-resistant strains (n = 77).

4.4.1 CA *Xaj* (n=115)

The presence absence matrix file this data was acquired from is `matrix_Xaj_115.xlsx`.

4.4.1.1 Pangenome Partitions

A pangenome graph was generated with `Gephi` using the `pangenomeGraph.gexf` as the input file. Nodes represent gene families (clusters of homologous genes across the input genomes). Edges connect gene families that are genomic neighbors (i.e., their sequences are adjacent in at least one genome). The graph is partitioned into categories—persistent, shell, and cloud—based on how frequently gene families appear across genomes and their neighborhood relationships. This structure resembles a "subway map" of genomic diversity, allowing you to visualize the core and variable components of a species' pangenome. The 'persistent,' 'shell,' and 'cloud' genomes constitute 69.95%, 19.14%, and 10.91% of the pangenome, respectively (4.4).

4.4.1.2 Copper Resistance-Related Genes

Various copper resistance-related genes have been characterized in *Xanthomonas* and in *Xaj*. These genes include: *copL*, *copA*, *copB*, *copC*, *copD*, and *copK* (Lee et al. (1994); cesbron2015; Marin et al. (2019); Assis et al. (2021)). The *copA* and *copC* genes encode periplasmic copper-binding proteins, conferring partial copper resistance in *P. syringae* (Marin et al. (2019)). In *X. citri*, Richard et al. (2017) reported plasmid-bore copper resistance genes termed *copABCD*, however in *Xaj* these genes are found on the chromosome (Lee et al. (1994); Giovanardi et al. (2016); Assis et al. (2021)). In both copper-sensitive (CuS) and copper-resistant (CuR) *Xanthomonas* strains, the chromosomal genes *cohL*, *cohA*, and *cohB*, involved in copper homeostasis, exhibit homology to cop resistance genes located on plasmids (Behlau et al. (2011); Behlau et al. (2016)).

The `PPanGGoLiN` analysis successfully identified copper resistance-related genes as well as the copper homeostatsis genes. The genes associated with copper resistance are *copL* family metal-binding regulatory protein, copper resistance system multicopper oxidase (*copA*), copper resistance protein B (*copB*), copper homeostasis periplasmic binding protein *copC*, copper homeostasis membrane protein *copD*, and *copK* family periplasmic copper-binding protein. Each of these six genes are present in the 'shell' or 'cloud' partitions and while the respective *copL*, *copA*, *copB* homeostasis genes are present in the 'persistent' genome and found in all 115 strains (4.4).

A total of 77 strains were found to encode copper resistance genes (*copA*, *copB*, *copC*, *copD*, and *copK*) in various combinations and copy number. These same 77 strains were also characterized as CuR based on the *in vitro* sensitivity assay.

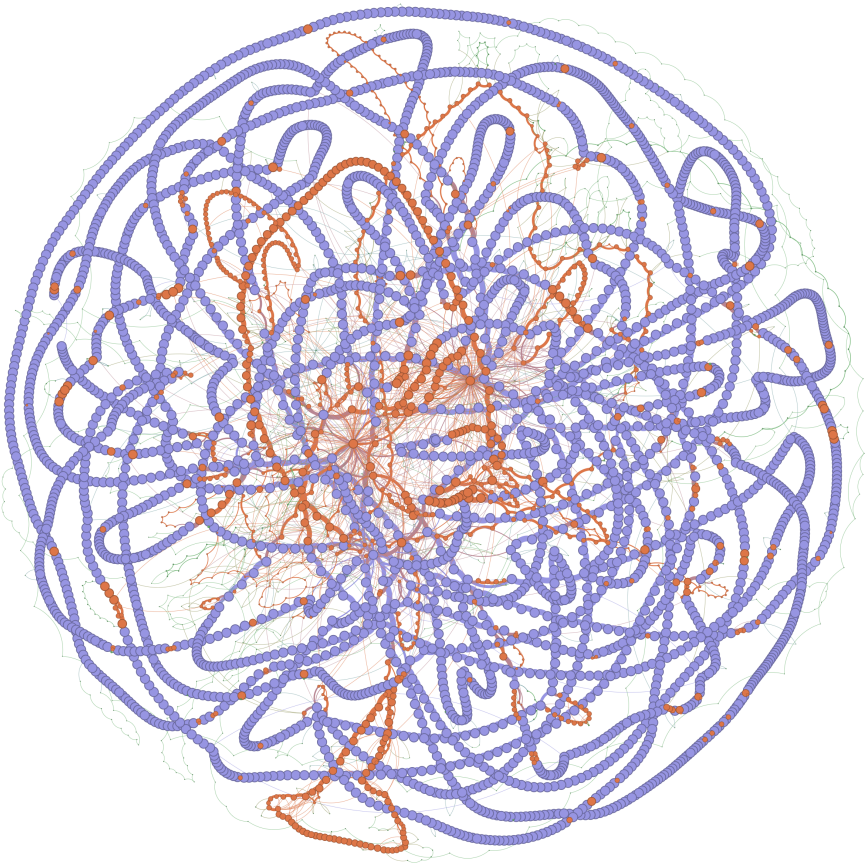


Figure 4.4: Pangenome graph of the 115 CA *Xaj* strains. 'Persistent': purple; 'shell': orange; 'cloud': green.

Table 4.4: Genes associated with copper resistance and homeostasis. The annotations have been shortened, see `matrix_Xaj_115.xlsx` tab `Copper-resistance` for the full annotations.

Gene (PPanGGoLiN ID)	Partition	Annotation	No. strains	No. seqs	Avg seq per strain
ACETXQ_000680	cloud	*copA*	3	3	1
ACETXQ_000681	cloud	*copB*	3	3	1
ACETXQ_000677	cloud	*copC*	3	3	1
ACETXQ_000676	cloud	*copD*	3	3	1
ACETZU_002141	cloud	*copA*	11	11	1
ACETY7_002282	cloud	*copB*	11	11	1
ACETY7_002278	cloud	*copC*	11	11	1
ACETY7_002277	cloud	*copD*	11	11	1
ACETY7_002276	cloud	*copK*	11	11	1
ACETYR_002169	shell	*copL*	59	107	1.81
ACETYY_002113	shell	*copL*	63	111	1.76
ACETZG_002299	shell	*copA*	63	111	1.76
ACETZG_002300	shell	*copB*	63	111	1.76
ACETZG_002304	shell	*copC*	63	111	1.76
ACETZG_002302	shell	*copD*	63	111	1.76
ACETZG_003906	persistent	*copL*	115	115	1
ACET0S_003856	persistent	*copA*	115	115	1
ACETZG_003908	persistent	*copB*	115	115	1

4.4.1.3 Type III Effectors

The presence/absence matrix (`matrix_Xaj_115.xlsx`) was used to identify type III effectors (T3Es) by searching for the term "tpye III secretion system effector" in the 'Annotations' column. I was able to identify 18 effectors using this method. After cross-referencing the tBLASTn & BLASTp results, I was able to identify 4 more that did not have "type III secretion system effector" in the annotation but were determined to be T3Es by Method 3 (3.6).

Only two T3Es were identified in the presence/absence matrix, *XopR* and *XopAH*, that were not captured by Method 3 (**LINK METHOD 3 RESULTS TABLE**).

- The XAJ283A *XopAH* gene (ACET0C_003701) was present in the raw `BLASTp` and `tBLASTn` outputs but exhibited ~28% identity with E-values of 5.11e-33 and 1.53e-36 to WP_011037263.1 (*XopAH*), respectively. I performed a web-based BLASTp with XAJ283A's *XopAH* gene ACET0C_003701 it yielded >99% ID to X. arboricola "XopAH/AvrB family type III secretion system effector" (WP_372159172.1,

WP_080591327.1, and WP_258077835.1) and to Xaj "type III effector protein AvrB3" (AKU50911.1). The reason that this gene was not captured in the BLAST analyses is likely because the reference *XopAH* WP_011037263.1 is a *X. campestris* protein and exhibited too much sequence variation to the query *Xaj* sequences.

- The XAJ283A *XopR* gene (ACET0C_000329) was present in the raw `BLASTp` and `tBLASTn` outputs but exhibited ~37% and ~44% identity with E-values of 2.12e-60 and 4.89e-36 to WP_011260681.1 (*XopR*), respectively. I performed a web-based BLASTp with XAJ283A's *XopR* gene ACET0C_000329 it yielded 100% and 99.76 % idenetity to *X. arboricola* "type III secretion system effector protein *XopR*" (WP_104538897.1) and to the *X. arboricola* pv. *corylina* "type III secretion system effector protein *XopR*" (WIX26666.1), respectively. The reason this gene was not captured in the BLAST analyses is likely because the reference *XopR* sequence (WP_011260681.1) is a MULTISPECIES *Xanthomonas XopR* sequence and exhibited too much sequence variation to the query *Xaj* sequences.

The respective protein sequences for XopR and XopAH in strain Xaj283A were subjected to a web-based BLASTp search and the results yielded (4.5).

Table 4.5: Type III effectors identified from the PPanGGoLiN presence/absence matrix (`matrix_Xaj_115.xlsx`).

Gene (PPanGGoLiN ID)	Partition	T3E	No. strains	No. sequences	Avg sequences per strain	Captured by Method 3?	Type
ACETYW_00216	cloud	*AvrBs1*	2	2	1	Yes	Variable
ACETYW_00218	cloud	*XopH1*	2	2	1	Yes	Variable
ACETZ0_03513	cloud	*XopG2*	14	14	1	Yes	Variable
ACETZG_00545	persistent	*AvrBs2*	115	115	1	Yes	Core
ACETZV_00338	persistent	*XopR*	115	115	1	No	Core
ACETZG_01290	persistent	*XopAV1*	115	115	1	Yes	Core
ACETZG_01291	persistent	*XopAV2*	115	115	1	Yes	Core
ACETZG_00577	persistent	*XopF1*	115	115	1	Yes	Core
ACETZG_00328	persistent	*XopK*	115	115	1	Yes	Core
ACETZG_00362	persistent	*XopK*	115	115	1	Yes	Core
ACETZG_01289	persistent	*XopQ*	115	115	1	Yes	Core
ACETZG_03209	persistent	*XopAW*	112	112	1	Yes	Variable
ACETZG_00745	persistent	*XopV*	115	115	1	Yes	Core
ACETZG_00656	persistent	*XopX*	115	115	1	Yes	Core
ACETZG_02419	persistent	*XopZ*	115	115	1	Yes	Core
ACETZG_03349	persistent	*XopL*	115	115	1	Yes	Core
ACETZG_00602	persistent	*XopM*	115	115	1	Yes	Core

Gene (PPanGGoLiN ID)	Partition	T3E	No. strains	No. sequences	Avg sequences per strain	Captured by Method 3?	Type
ACETZG_00[illegible]	persistent	*XopAZ*	115	115	1	Yes	Core
ACETZG_00[illegible]	shell	*XopG3*	110	110	1	Yes	Variable
ACETY6_03[illegible]	shell	*XopAI*	48	48	1	Yes	Variable
ACETYR_03[illegible]	shell	*XopAH*	78	79	1.01	No	Variable
ACETZG_00[illegible]	shell	*XopE4*	53	53	1	Yes	Variable

4.4.1.4 Modules, Spots, and RGPs (LEFT OFF HERE!!!)

Modules

- Definition: Modules in PPanGGOLiN are conserved groups of genes, typically from the variable (shell and cloud) genome, that tend to co-occur and are co-localized across multiple genomes in the pangenome. These are often hypothesized to represent functional units that are gained or lost together.
- How They're Generated: Modules are predicted using the panModule method, which analyzes patterns of gene co-occurrence and genomic neighborhood within RGPs. The process involves identifying clusters of genes that consistently appear together in the variable regions of the pangenome.

Spots

- Definition: Spots, or Spots of Insertion, are groups of RGPs from different genomes that share conserved flanking persistent genes, indicating a common insertion site in the pangenome. They represent hotspots of genomic variability.
- How They're Generated: After RGPs are identified, the spot command groups them into Spots based on shared genomic context (persistent gene borders). This clustering reflects where RGPs tend to integrate across the pangenome.

RGPs (Regions of Genomic Plasticity)

- Definition: RGPs are genomic regions in individual genomes that consist of genes from the shell and cloud partitions (i.e., non-persistent genes). These regions are typically associated with horizontal gene transfer (HGT) and often correspond to Genomic Islands (GIs).
- How They're Generated: The panRGP method identifies RGPs by scanning each genome and detecting clusters of consecutive shell or cloud genes, using a score-based algorithm to define boundaries. Persistent genes flanking these regions help delineate them.

Interconnection

- Modules, RGPs, and Spots are interconnected: Modules are often subsets of genes within RGPs, and Spots group RGPs across genomes. The –spot_modules option ties these together by associating Modules with specific Spots and RGPs, enabling a deeper understanding of how conserved gene clusters relate to variable genomic regions.
- Purpose: Together, these features allow PPanGGOLiN to depict microbial diversity by partitioning the pangenome into persistent, shell, and cloud components, then analyzing the variable regions for patterns of conservation and plasticity.

4.5 Phylogenetic Analyses (NEED TO FINISH)

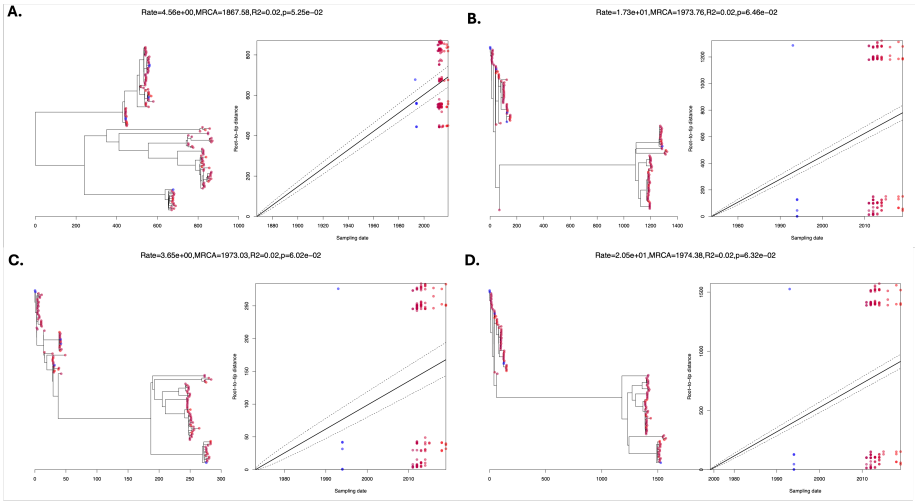


Figure 4.5: Root-to-tip regression analysis on CA_XAJ using BactDating. **A.** ClonalFrameML. **B.** Gubbins. **C.** ClonalFrameML and IQtree. **D.** ClonalFrameML and Gubbins

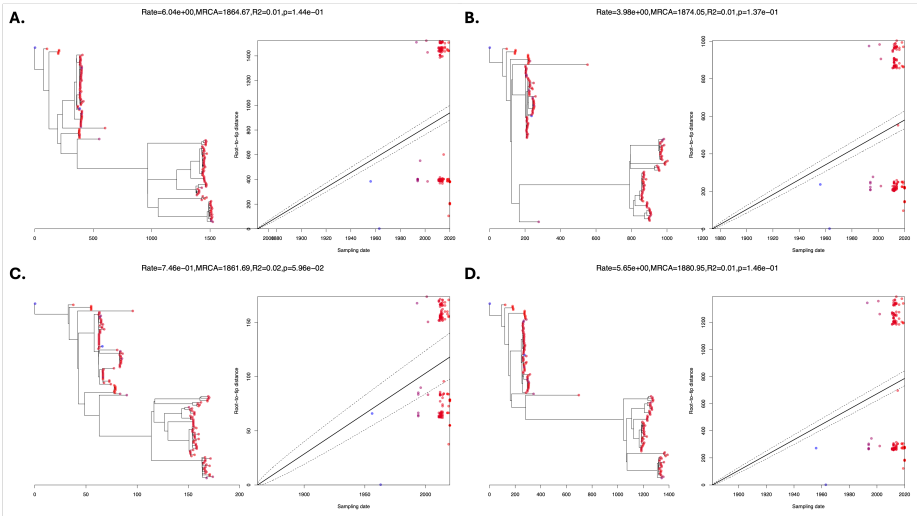


Figure 4.6: Root-to-tip regression analysis on CA_XAJ_NCBI_24 using BactDating. **A.** ClonalFrameML. **B.** Gubbins. **C.** ClonalFrameML and IQtree. **D.** ClonalFrameML and Gubbins

Chapter 5

Scripts

5.1 Read Processing

5.1.1 FastP.sh

```
#!/bin/bash -l

#SBATCH --nodes=1
#SBATCH --cpus-per-task=4
#SBATCH--mem=16G
#SBATCH --time=7-00:00:00
##SBATCH --output=my.stdout
#SBATCH --mail-user=ashan007@ucr.edu
#SBATCH --mail-type=ALL
#SBATCH --job-name="FastP"
#SBATCH -p batch

# Setting sample files
SAMPFILE=/bigdata/adaskaveglab/ashan007/Scripts/Strain_CSV/Strains.csv
# Setting Output directory variables
FILTERED=/bigdata/adaskaveglab/ashan007/Filtered_Reads
REPORTS=/bigdata/adaskaveglab/ashan007/FastP_Reports
IFS=,
# Run FastP v.0.23.4
tail -n +2 $SAMPFILE | while read SID STRAIN READ1 READ2 ORGANISM
do
  /bigdata/adaskaveglab/ashan007/Programs/fastp \
  -i $READ1 -I $READ2 \
  -o $FILTERED/$SID.F.1.fq.gz -O $FILTERED/$SID.F.2.fq.gz \
  -q 30 \
```

```
  --detect_adapter_for_pe \
  -h $REPORTS/$SID.html
done
```

5.2 De Novo Genome Assembly

5.2.1 Step1.Assemble.sh

```
#!/bin/bash -l

#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=16
#SBATCH --mem=64G
#SBATCH --time=14-00:00:00
##SBATCH --output=my.stdout
#SBATCH --mail-user=ashan007@ucr.edu
#SBATCH --mail-type=ALL
#SBATCH --job-name="Step 1"
#SBATCH -p batch
#SBATCH --array 1-140

###############################################################################################
################################### Load Modules ##############################################
###############################################################################################
module load quast/5.1.0rc1
###############################################################################################
################################## Define Variables ###########################################
###############################################################################################
# Defining Step Directories
STEP1=/bigdata/adaskaveglab/ashan007/Assembly_Workflow_PlatanusB/Step1_Assembly/Results
STATS=/bigdata/adaskaveglab/ashan007/Assembly_Workflow_PlatanusB/Step1_Assembly/Stats
SAMPFILE=/bigdata/adaskaveglab/ashan007/Scripts/Strain_CSV/Assembly_Workflow_Platanus_I
SCAFFOLD=/bigdata/adaskaveglab/ashan007/Scripts/Assembly_Workflow_PlatanusB

# Defining Program paths
PLATANUS=/bigdata/adaskaveglab/ashan007/Programs/Platanus_B-1.3.2
###############################################################################################
################################## Define Array Vars ##########################################
###############################################################################################
CPU=1

if [ $SLURM_CPUS_ON_NODE ]; then
 CPU=$SLURM_CPUS_ON_NODE
```

```
fi

N=${SLURM_ARRAY_TASK_ID}

if [ ! $N ]; then
 N=$1
fi

if [ ! $N ]; then
 echo "need to provide a number by --array slurm or on the cmdline"
 exit
fi
################################################################################################################
#                                                    Begin Pipeline
################################################################################################################
IFS=,
tail -n +2 "$SAMPFILE" | sed -n "${N}p" | while read -r SID STRAIN READ1 READ2 ORGANISM; do
  ##############################################################################################################
  ##############################################################################################################
  #                                        Step 2 Generate Assembly via Platanus_B
  ##############################################################################################################
  ##############################################################################################################
  if [ -d "$STEP1/$STRAIN" ]; then
    echo "Skipping $STRAIN - Directory already exists"
    continue  # Skip to the next iteration if directory exists
  else
    # Create the directory and check if the command succeeds
    if mkdir -p "$STEP1/$STRAIN"; then
      echo "Processing $STRAIN via Platanus_B... STEP 1"

      $PLATANUS/./platanus_b assemble \
      -o "$STEP1/$STRAIN/$STRAIN" \
      -f "$READ1" "$READ2" \
      -t 16 -m 64
    else
      echo "Error: Failed to create directory "$STEP1/$STRAIN""
    fi
  fi
  ##############################################################################################################
  ##############################################################################################################
  #                                        Generate Assembly Statistics via QUAST
  ##############################################################################################################
  ##############################################################################################################

  if [ -d "$STATS/Contigs/$STRAIN" ]; then
```

```
    echo "Skipping $STRAIN - Directory already exists"
    continue  # Skip to the next iteration if directory exists
  else
    # Create the directory and check if the command succeeds
    if mkdir -p "$STATS/Contigs/$STRAIN"; then
      echo "Processing $STRAIN via QUAST... STEP 2"
      # Contigs
      quast.py "$STEP1/$STRAIN/${STRAIN}_contig.fa" -o "$STATS/Contigs/$STRAIN"
    else
      echo "Error: Failed to create directory "$STATS/Contigs/$STRAIN""
    fi
  fi
done
```

5.2.2 Summarize_ANI.sh

```
#!/bin/bash

# Initialize the output file with the header
echo -e "Sample\tAssembly\tSubmitted Organism\tPredicted Organism\tSubmitted Type\tStat

# Loop over each subdirectory
for dir in */; do
    # Check if the directory contains the ani-tax-report.txt file
    if [[ -f "${dir}ani-tax-report.txt" ]]; then
        # Extract the relevant lines from the ani-tax-report.txt file
        assembly=$(sed -n '1p' "${dir}ani-tax-report.txt" | cut -d':' -f2 | xargs)
        submitted_organism=$(sed -n '2p' "${dir}ani-tax-report.txt" | cut -d':' -f2 | x
        predicted_organism=$(sed -n '3p' "${dir}ani-tax-report.txt" | cut -d':' -f2 | x
        submitted_type=$(sed -n '4p' "${dir}ani-tax-report.txt" | cut -d':' -f2 | xargs
        status=$(sed -n '5p' "${dir}ani-tax-report.txt" | cut -d':' -f2 | xargs)
        confidence=$(sed -n '6p' "${dir}ani-tax-report.txt" | cut -d':' -f2 | xargs)

        # Capture the ANI value from the first data row under the dashed line followin
        best_hit=$(awk '/^-------/{getline; print $1; exit}' "${dir}ani-tax-report.txt"

        # Get the directory name without the trailing slash (which is the sample name)
        sample=${dir%/}

        # Append the data to the summary table
        echo -e "${sample}\t${assembly}\t${submitted_organism}\t${predicted_organism}\t
    fi
done

echo "ANI summary table has been created in ani_summary_table.tsv"
```

5.2.3 Summarize_CheckM.sh

```
#!/bin/bash

# Initialize the output file with the header
echo -e "Sample\tBin Id\tMarker lineage\t# genomes\t# markers\t# marker sets\t0\t1\t2\t3\t4\t5+\t

# Loop over each subdirectory
for dir in */; do
    # Check if the directory contains the checkm.txt file
    if [[ -f "${dir}checkm.txt" ]]; then
        # Extract the second line (data) from the checkm.txt file
        data=$(sed -n '2p' "${dir}checkm.txt")
        # Get the directory name without the trailing slash (which is the sample name)
        sample=${dir%/}
        # Append the sample name and the data to the final table
        echo -e "${sample}\t${data}" >> checkm_summary_table.tsv
    fi
done

echo "CheckM summary table has been created in checkm_summary_table.tsv"
```

5.2.4 Summarize_Genbank.py

```
#!/usr/bin/env python
# coding: utf-8

import os
import re
import pandas as pd

def extract_genome_annotation_data(file_path):
    try:
        with open(file_path, 'r', encoding='utf-8', errors='ignore') as file:
            content = file.read()
    except UnicodeDecodeError:
        print(f"Failed to decode file: {file_path}")
        return None

    # Find the section starting with "#Genome-Annotation-Data-START##"
    start_index = content.find("#Genome-Annotation-Data-START##")
    if start_index == -1:
        print(f"Section not found in file: {file_path}")
```

```
        return None  # Section not found

    # Extract the section
    section = content[start_index:]

    # Define the keys to extract
    keys = [
        "Genes (total)", "CDSs (total)", "Genes (coding)",
        "CDSs (with protein)", "Genes (RNA)", "rRNAs",
        "complete rRNAs", "partial rRNAs", "tRNAs",
        "ncRNAs", "Pseudo Genes (total)", "CDSs (without protein)"
    ]

    data = {}
    for key in keys:
        match = re.search(rf"{re.escape(key)}\s*::\s*(.+)", section)
        if match:
            value = match.group(1).strip()
            # Convert values to int if possible
            try:
                value = int(value.replace(',', ''))
            except ValueError:
                pass
            data[key] = value
        else:
            data[key] = None  # Key not found or no value
            print(f"Key '{key}' not found in section of file: {file_path}")

    return data

def process_genbank_files(base_directory):
    result = []
    # Iterate over each subdirectory
    for root, dirs, files in os.walk(base_directory):
        for filename in files:
            if filename.endswith(".gbk"):
                sample_name = os.path.basename(root)  # Get the subdirectory name as s
                file_path = os.path.join(root, filename)
                data = extract_genome_annotation_data(file_path)
                if data:
                    # Add the sample name to the data
                    data['Sample'] = sample_name
                    result.append(data)
                else:
                    print(f"No data extracted from file: {file_path}")
```

```
    return result

def main():
    base_directory = "/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step4_Annotate/
    data = process_genbank_files(base_directory)

    if data:
        # Convert the list of dictionaries to a pandas DataFrame
        df = pd.DataFrame(data)

        # Reorder columns to place 'Sample' at the beginning
        cols = ['Sample'] + [col for col in df.columns if col != 'Sample']
        df = df[cols]

        # Save the DataFrame to a CSV file
        df.to_csv('Genome_Annotation_Summary.csv', index=False)

        print("Genome annotation data has been extracted and saved to Genome_Annotation_Summary.c
    else:
        print("No genome annotation data found in the provided files.")

if __name__ == "__main__":
    main()
```

5.2.5 Generate_RCP.sh

```
#!/bin/bash

# Set the path to your strains file
STRAINS_FILE="/Volumes/AS_Postdoc/Postdoc2024/Strain_Datasets/XAJ_Strains.RCP.csv"

# Set the path to your template file
TEMPLATE_FILE="/Volumes/AS_Postdoc/Postdoc2024/Ragout/Template_RCP_Files/Platanus_B/XAJ_Template.

# Directory to save the output .rcp files
OUTPUT_DIR="//Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step2_Scaffold/rcp_files
mkdir -p $OUTPUT_DIR

# Skip the header and loop through each strain in the strains file
tail -n +2 "$STRAINS_FILE" | while IFS=, read -r strain; do
    # Replace $STRAIN with the actual strain name in the template file and ensure .fa stays attac
    sed "s/\$STRAIN/$strain/g" "$TEMPLATE_FILE" | sed "s/$strain \.fa/$strain.fa/g" > "$OUTPUT_DI
    dos2unix "$OUTPUT_DIR/$strain.rcp"
```

```
    echo "Generated: $OUTPUT_DIR/$strain.rcp"
done
```

5.2.6 Step2.Scaffold.sh

```
#!/bin/bash -l

#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=16
#SBATCH --mem=64G
#SBATCH --time=14-00:00:00
##SBATCH --output=my.stdout
#SBATCH --mail-user=ashan007@ucr.edu
#SBATCH --mail-type=ALL
#SBATCH --job-name="Pipeline"
#SBATCH -p batch
#SBATCH --array 1-140

################################################################################
################################## NOTE!!!!!!!! ################################
################################################################################
# I renamed the chromosomes in the refernce fasta files used for the scaffolding
# IVIA1317 and 15_088) becuase Ragout will use their ID to name our strains. I do
# not want out population to have the same NCBI assembly ID for obvious reasons so
# I renamed the chromosome to chr1 for both. I only modified those two fastas
# located: /bigdata/adaskaveglab/ashan007/Assembly_Workflow/Step3_Scaffold/References
################################################################################
################################## Load Modules ################################
################################################################################
# module load ragout/v2.3
# module load quast/5.1.0rc1
################################################################################
################################ Define Variables ##############################
################################################################################
# Defining Step Directories
SAMPFILE=/bigdata/adaskaveglab/ashan007/Scripts/Strain_CSV/Assembly_Workflow_Platanus_

# IVIA as reference with contigs
IVIACONRES=/bigdata/adaskaveglab/ashan007/Assembly_Workflow_PlatanusB/Step2_Scaffold/IV
IVIACONSTATS=/bigdata/adaskaveglab/ashan007/Assembly_Workflow_PlatanusB/Step2_Scaffold/
IVIACONRECIPEDIR=/bigdata/adaskaveglab/ashan007/Assembly_Workflow_PlatanusB/Step2_Scaf

################################################################################
################################ Define Array Vars #############################
```

```
#################################################################################
CPU=1

if [ $SLURM_CPUS_ON_NODE ]; then
 CPU=$SLURM_CPUS_ON_NODE
fi

N=${SLURM_ARRAY_TASK_ID}

if [ ! $N ]; then
 N=$1
fi

if [ ! $N ]; then
 echo "need to provide a number by --array slurm or on the cmdline"
 exit
fi
#########################################################################################################
#                                                  Begin Pipeline
#########################################################################################################
IFS=,
tail -n +2 "$SAMPFILE" | sed -n "${N}p" | while read -r SID STRAIN READ1 READ2 ORGANISM; do

  ###############################################################################################
  ###############################################################################################
  #                                          Step 4 Scaffold Contigs via Ragout
  ###############################################################################################
  ###############################################################################################
  module load ragout/v2.3
# Contigs with IVIA as the reference
    if [ -d "$IVIACONRES/$STRAIN" ]; then
    echo "Skipping $STRAIN - Directory already exists"
    continue  # Skip to the next iteration if directory exists
  else
    # Create the directory and check if the command succeeds
    if mkdir -p "$IVIACONRES/$STRAIN"; then
      ragout -o "$IVIACONRES/$STRAIN" -s sibelia --refine -t 16 "$IVIACONRECIPEDIR/${STRAIN}.rcp"
    else
      echo "Error: Failed to create directory $IVIACONRES/$STRAIN"
    fi
  fi
  module unload ragout/v2.3
  ###############################################################################################
  ###############################################################################################
  #                                      Step 3 Generate Assembly Statistics via QUAST
```

```
  ##############################################################################################
  ##############################################################################################
  module load quast/5.1.0rc1
# Contigs with IVIA as the reference
    if [ -d "$IVIACONSTATS/$STRAIN" ]; then
    echo "Skipping $STRAIN - Directory already exists"
    continue  # Skip to the next iteration if directory exists
  else
    # Create the directory and check if the command succeeds
    if mkdir -p "$IVIACONSTATS/$STRAIN"; then
      echo "Processing $STRAIN via QUAST... STEP 2"
      # SPAdes
      quast.py "$IVIACONRES/$STRAIN/${STRAIN}_scaffolds.fasta" -o "$IVIACONSTATS/$STRA

    else
      echo "Error: Failed to create directory $IVIACONSTATS/$STRAIN"
    fi
  fi

  module unload quast/5.1.0rc1
done
```

5.2.7 Step3.Polish.sh

```
#!/bin/bash -l

#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=16
#SBATCH --mem=64G
#SBATCH --time=14-00:00:00
##SBATCH --output=my.stdout
#SBATCH --mail-user=ashan007@ucr.edu
#SBATCH --mail-type=ALL
#SBATCH --job-name="Step3"
#SBATCH -p batch
#SBATCH --array 1-140

##############################################################################################
################################### Load Modules #############################################
##############################################################################################
module load pilon/1.24
module load samtools/1.19.2
module load bwa-mem2/2.2.1
##############################################################################################
```

```
############################## Define Variables ################################
################################################################################
# Defining Step Directories
SAMPFILE=/bigdata/adaskaveglab/ashan007/Scripts/Strain_CSV/Assembly_Workflow_Platanus_B.csv
IVIACONRES=/bigdata/adaskaveglab/ashan007/Assembly_Workflow_PlatanusB/Step3_Polish/IVIA_Contig_Re
IVIACONGENO=/bigdata/adaskaveglab/ashan007/Assembly_Workflow_PlatanusB/Step2_Scaffold/IVIA_Contig
IVIA_CONBAMDIR=/bigdata/adaskaveglab/ashan007/Assembly_Workflow_PlatanusB/Step3_Polish/IVIA_Conti
IVIA_CONINDEXDIR=/bigdata/adaskaveglab/ashan007/Assembly_Workflow_PlatanusB/Step3_Polish/IVIA_Cor

# Defining Program paths
PILONJAR=/opt/linux/rocky/8.x/x86_64/pkgs/pilon/1.24/env/share/pilon-1.24-0/pilon.jar
################################################################################
############################## Define Array Vars ###############################
################################################################################
CPU=1

if [ $SLURM_CPUS_ON_NODE ]; then
 CPU=$SLURM_CPUS_ON_NODE
fi

N=${SLURM_ARRAY_TASK_ID}

if [ ! $N ]; then
 N=$1
fi

if [ ! $N ]; then
 echo "need to provide a number by --array slurm or on the cmdline"
 exit
fi
################################################################################################################
#                                                  Begin Pipeline
################################################################################################################
IFS=,
tail -n +2 "$SAMPFILE" | sed -n "${N}p" | while read -r SID STRAIN READ1 READ2 ORGANISM SEQTK; do

      ########################################
      #                 BWA                  #
      ########################################
  if [ -d "$IVIACONRES/$STRAIN" ]; then
    echo "Skipping $STRAIN - Directory already exists"
    continue  # Skip to the next iteration if directory exists
  else
    # Create the directory and check if the command succeeds
    if mkdir -p "$IVIACONRES/$STRAIN"; then
```

```
      echo "Processing $STRAIN via BWA..."

      bwa-mem2 index -p "$IVIA_CONINDEXDIR/${STRAIN}" "$IVIACONGENO/${STRAIN}_scaffolds
      bwa-mem2 mem -t 16 "$IVIA_CONINDEXDIR/${STRAIN}" \
      "$READ1" "$READ2" | samtools view -b | samtools sort -@ 16 -o "$IVIA_CONBAMDIR/$
      samtools index "$IVIA_CONBAMDIR/${STRAIN}.bam"

      ########################################
      #                Pilon                 #
      ########################################
      java -jar -Xmx64G $PILONJAR --genome "$IVIACONGENO/${STRAIN}_scaffolds.fasta" \
      --bam "$IVIA_CONBAMDIR/${STRAIN}.bam" \
      --output "${STRAIN}_Pilon" \
      --outdir "$IVIACONRES/$STRAIN"

    else
      echo "Error: Failed to create directory $IVIACONRES/$STRAIN"
    fi
  fi
done
```

5.2.8 Step3.CheckM.sh

```
#!/bin/bash -l

#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH--mem=64G
#SBATCH -c 16
#SBATCH --time=1-00:00:00
##SBATCH --output=my.stdout
#SBATCH --mail-user=ashan007@ucr.edu
#SBATCH --mail-type=ALL
#SBATCH --job-name="CheckM"
#SBATCH -p batch

################################################################################
################################## Load Modules ################################
################################################################################
module load checkm/1.2.1
################################################################################
################################ Define Variables ##############################
################################################################################
# Defining Step Directories
```

```
OUTDIR=/bigdata/adaskaveglab/ashan007/Assembly_Workflow_PlatanusB/Step3_Polish/CheckM
IVIACON=/bigdata/adaskaveglab/ashan007/Assembly_Workflow_PlatanusB/Step3_Polish/Fastas/IVIA_Conti
################################################################################
################################## Run CheckM ##################################
################################################################################
# Pilon polished Ragout congigs using IVIA as reference
checkm taxonomy_wf -t 16 genus Xanthomonas -x fasta "$IVIACON" "$OUTDIR/CheckM_Tax_Contigs_IVIA"
```

5.2.9 Generate_Generic_YAML.sh

```
#!/bin/bash

# Define the input file
input_file="Reference_Data_For_YAML.txt"

# Define the directories
assembly_dir="/path/to/dir"
submol_yaml_dir="/path/to/dir"

# Read the input file line by line
tail -n +2 "$input_file" | while IFS=$'\t' read -r sra_accession study bioproject biosample strai

    # Generate the YAML file name based on the strain
    yaml_file="generic_yaml_files/${strain}.generic.yaml"

    # Create the YAML file
    cat <<EOL > "$yaml_file"
fasta:
    class: File
    location: ${assembly_dir}/${strain}.fa
submol:
    class: File
    location: ${submol_yaml_dir}/${strain}.submol.yaml
EOL

    echo "Generated $yaml_file"

done < "$input_file"
```

5.2.10 Generate_Submol_YAML.sh

```
#!/bin/bash

# Define the input file
```

```
input_file="Reference_Data_For_YAML.txt"

# Create the directory for YAML files
mkdir -p submol_yaml_files

# Read the input file line by line, skipping the header
tail -n +2 "$input_file" | while IFS=$'\t' read -r sra_accession study bioproject biosa

    # Define the output file name based on the strain
    output_file="submol_yaml_files/${strain}.submol.yaml"

    # Write the YAML content to the output file
    cat <<EOL > "$output_file"
organism:
    genus_species: '$genus_species'
    strain: '$strain'
contact_info:
    last_name: 'Shands'
    first_name: 'Aidan'
    email: 'ashan007@ucr.edu'
    organization: 'University of California, Riverside'
    department: 'Microbiology and Plant Pathology'
    street: '900 University Ave'
    city: 'Riverside'
    state: 'CA'
    postal_code: '92521'
    country: 'USA'
authors:
    - author:
        last_name: 'Shands'
        first_name: 'Aidan'
        middle_initial: 'C'
bioproject: '$bioproject'
biosample: '$biosample'
locus_tag_prefix: '$locus_tag'
sra:
    - accession: '$sra_accession'
EOL

    echo "Generated $output_file"

done
```

5.2.11 PGAP.sh

```
#!/bin/bash -l

# PGAP version 2024-07-18.build7555 is up to date.
# Defining Variables
SAMPFILE=PGAP.csv
PGAPPATH=/Users/aidanshands/Desktop/PGAP
YAMLDIR=/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step4_Annotate/Step2_Annotate

IFS=,  # Set the field separator for CSV parsing

# Use process substitution to avoid subshell issues
while read -r SID STRAIN; do
  # Debug: Print the values being read
  echo "Read: SID='$SID', STRAIN='$STRAIN'"

  # Check if the output directory already exists
  if [ -d "$STRAIN" ]; then
    echo "Skipping $STRAIN: Output directory already exists."
    continue  # Skip to the next strain
  fi

  # Run PGAP for the current strain
  echo "Processing $STRAIN..."
  "$PGAPPATH/pgap.py" -o "$STRAIN" --no-self-update -n --ignore-all-errors --taxcheck "$YAMLDIR/$

  # Debug: Check the exit status of the PGAP command
  if [ $? -ne 0 ]; then
    echo "Error: PGAP failed for $STRAIN. Continuing with the next sample."
    continue  # Don't stop the loop if there's an error
  fi

done < <(tail -n +2 "$SAMPFILE")

# Reset IFS (optional but good practice)
unset IFS
```

5.2.12 Step4.Annotate.sh

```
#!/bin/bash -l
# Loop to process the Xaj and Xap populations
while true; do
    echo "Starting PGAP.sh at $(date)" >> pgap_log.txt
    bash PGAP.sh
```

```
    echo "Completed PGAP.sh at $(date)" >> pgap_log.txt
    echo "Waiting 10 minutes..."
    sleep 600  # 600 seconds = 10 minutes
done
```

5.3 Pangenome

5.3.1 PPanggolin_TSV.sh

This was used for the CA_XAP_and_NCBI22 run but the same was applied to all runs.

```
#!/bin/bash

# Set the directory where your .gbk/.gbff files are located
directory="/bigdata/adaskaveglab/ashan007/Assembly_Workflow_PlatanusB/Step5_Analyses/G

# Create or clear the output TSV file
output_file="CA_XAP_and_NCBI22.tsv"
> "$output_file"

# Loop through all .gbk and .gbff files in the directory
for file in "$directory"/*.gbk; do
  # Extract the strain name (the filename without the extension)
  strain=$(basename "$file" | sed 's/\.[^.]*$//')

  # Write the strain name and absolute path to the output TSV file
  echo -e "$strain\t$file" >> "$output_file"
done

echo "TSV file generated: $output_file"
```

5.3.2 PPanggolin.sh

```
#!/bin/bash -l

#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH--mem=64G
#SBATCH -c 16
#SBATCH --time=1-00:00:00
##SBATCH --output=my.stdout
#SBATCH --mail-user=ashan007@ucr.edu
#SBATCH --mail-type=ALL
```

```
#SBATCH --job-name="ppanggolin"
#SBATCH -p batch

conda activate ppanggo
# PPanGGOLiN version:
PARENTDIR=/bigdata/adaskaveglab/ashan007/Assembly_Workflow_PlatanusB/Step5_Analyses/Pangenome/Ppa
# Define the list of subdirectories
subdir_list=("CA_XAJ" "CA_XAJ_and_NCBI26" "CA_XAJ_XAP" "CA_XAJ_XAP_and_NCBI_XAJ_XAP" "CA_XAJ_XAP_

# Loop over each subdirectory
for subdir in "${subdir_list[@]}"; do
    echo "Processing $subdir..."

    # Change to the specific subdirectory
    cd "$PARENTDIR/$subdir" || {
        echo "Failed to enter $PARENTDIR/$subdir. Skipping...";
        continue;
    }

    # Execute the ppanggolin command
    ppanggolin all --anno "$subdir.tsv" -c 16

    # Return to the parent directory
    cd - || exit
done
```

5.4 Phylogenetic Analyses

5.4.1 Parsnp_Final.sh

```
#!/bin/bash -l

#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH--mem=32G
#SBATCH --time=2-00:00:00
##SBATCH --output=my.stdout
#SBATCH --mail-user=ashan007@ucr.edu
#SBATCH --mail-type=ALL
#SBATCH --job-name="Parsnp"
#SBATCH -p intel
#SBATCH --array 1-8
```

```
# Activate parsnp v2.0.6
conda activate parsnp

# Setting sample file
SAMPFILE=/bigdata/adaskaveglab/ashan007/Scripts/Strain_CSV/Parsnp_Final.csv
XAJREF=/bigdata/adaskaveglab/ashan007/Assembly_Workflow_PlatanusB/Step5_Analyses/Genom
XAPREF=/bigdata/adaskaveglab/ashan007/Assembly_Workflow_PlatanusB/Step5_Analyses/Genom
CPU=1

if [ $SLURM_CPUS_ON_NODE ]; then
 CPU=$SLURM_CPUS_ON_NODE
fi

N=${SLURM_ARRAY_TASK_ID}

if [ ! $N ]; then
 N=$1
fi

if [ ! $N ]; then
 echo "need to provide a number by --array slurm or on the cmdline"
 exit
fi

IFS=,
tail -n +2 $SAMPFILE | sed -n ${N}p | while read RUN GENOMES DIRECTORY
do
  if [ -d $DIRECTORY ]; then
    echo "Skipping $RUN - Directory $DIRECTORY already exists"
  fi
  if [[ ! -d $DIRECTORY ]]; then
    mkdir $DIRECTORY
    echo "Created directory $DIRECTORY and starting $RUN"
    # conditional statement for the outgroup samples
    if [[ "$RUN" == "CA_XAP" || "$RUN" == "CA_XAP_and_NCBI22" ]]; then
      # Xap as reference for CA_XAP and CA_XAP_and_NCBI22 Runs
      parsnp -p 8 -u --vcf -d $GENOMES -r $XAPREF -o $DIRECTORY
    else
      # Xaj as reference for remainders
      parsnp -p 8 -u --vcf -d $GENOMES -r $XAJREF -o $DIRECTORY
    fi
  fi
done
```

5.4.2 HarvestTools_Final.sh

```
#!/bin/bash -l

#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH--mem=32G
#SBATCH --time=02:00:00
##SBATCH --output=my.stdout
#SBATCH --mail-user=ashan007@ucr.edu
#SBATCH --mail-type=ALL
#SBATCH --job-name="Parsnp"
#SBATCH -p short
#SBATCH --array 1-8

# Setting sample file
SAMPFILE=/bigdata/adaskaveglab/ashan007/Scripts/Strain_CSV/Parsnp_Final.csv
CPU=1

if [ $SLURM_CPUS_ON_NODE ]; then
 CPU=$SLURM_CPUS_ON_NODE
fi

N=${SLURM_ARRAY_TASK_ID}

if [ ! $N ]; then
 N=$1
fi

if [ ! $N ]; then
 echo "need to provide a number by --array slurm or on the cmdline"
 exit
fi

IFS=,
tail -n +2 $SAMPFILE | sed -n ${N}p | while read RUN GENOMES DIRECTORY
do
  if [ -d $DIRECTORY ]; then
    echo "Skipping $RUN - Directory $DIRECTORY already exists"
  fi
  if [[ ! -d $DIRECTORY ]]; then
    mkdir $DIRECTORY
    echo "Created directory $DIRECTORY and starting $RUN"
  fi
```

```
    /bigdata/adaskaveglab/ashan007/Programs/harvesttools-Linux64-v1.3/./harvesttools -x
done
```

5.4.3 IQTree2_MF_Parsnp_CFML_Final.sh

```
#!/bin/bash -l

#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=32G
#SBATCH --time=14-00:00:00
##SBATCH --output=my.stdout
#SBATCH --mail-user=ashan007@ucr.edu
#SBATCH --mail-type=ALL
#SBATCH --job-name="IQtree2"
#SBATCH -p batch
#SBATCH --array 1-8


# Load modules
module load IQ-TREE/2.2.2.6

# Before running IQtree2 I removed the .fna.ref and .fna extensions from the alignment
# # Check the names before
# grep '^>' CA_XAP_CFML.filtered.fasta | sed 's/^>//' > CA_XAP_CFML.filtered.names.txt
# # Remove the extensions
# sed 's/\.fna\.ref\|\.fna//g' CA_XAP_CFML.filtered.fasta > CA_XAP_CFML.renamed.filter
# # Check the names after
# grep '^>' CA_XAP_CFML.renamed.filtered.fasta | sed 's/^>//' > CA_XAP_CFML.renamed.fi

# Cnce I confirmed that this worked I only ran the following for each file in their re
# sed -i 's/\.fna\.ref\|\.fna//g' CA_XAJ_CFML.filtered.fasta
# sed -i 's/\.fna\.ref\|\.fna//g' CA_XAJ_and_NCBI26_CFML.filtered.fasta
# sed -i 's/\.fna\.ref\|\.fna//g' CA_XAJ_XAP_CFML.filtered.fasta
# sed -i 's/\.fna\.ref\|\.fna//g' CA_XAJ_XAP_and_NCBI_XAJ_XAP_CFML.filtered.fasta
# sed -i 's/\.fna\.ref\|\.fna//g' CA_XAJ_XAP_and_NCBI_XAJ_XAP_and_Outgroup_CFML.filter
# sed -i 's/\.fna\.ref\|\.fna//g' CA_XAP_CFML.filtered.fasta
# sed -i 's/\.fna\.ref\|\.fna//g' CA_XAP_and_NCBI22_CFML.filtered.fasta

# Setting sample file
SAMPFILE=/bigdata/adaskaveglab/ashan007/Scripts/Strain_CSV/IQtree2_Parsnp_CFML_Final.cs

CPU=1
```

```
if [ $SLURM_CPUS_ON_NODE ]; then
 CPU=$SLURM_CPUS_ON_NODE
fi

N=${SLURM_ARRAY_TASK_ID}

if [ ! $N ]; then
 N=$1
fi

if [ ! $N ]; then
 echo "need to provide a number by --array slurm or on the cmdline"
 exit
fi

IFS=,
tail -n +2 $SAMPFILE | sed -n ${N}p | while read RUN ALIGNMENT MODEL MFDIRECTORY IQDIRECTORY OUTG
do
  if [ -d "$MFDIRECTORY" ]; then
    echo "Skipping $RUN - Directory $MFDIRECTORY already exists"
  fi
  if [[ ! -d "$MFDIRECTORY" ]]; then
    mkdir -p "$MFDIRECTORY"
    echo "Created directory $MFDIRECTORY and starting $RUN"
  # conditional statement for the outgroup samples
    iqtree2 -s "$ALIGNMENT" -mem 32G -T AUTO -m MFP --prefix "$MFDIRECTORY/MFout"
  fi
done
```

5.4.4 IQTree2_Parsnp_CFML_Final.sh

```
#!/bin/bash -l

#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=32G
#SBATCH --time=14-00:00:00
##SBATCH --output=my.stdout
#SBATCH --mail-user=ashan007@ucr.edu
#SBATCH --mail-type=ALL
#SBATCH --job-name="IQtree2"
#SBATCH -p batch
#SBATCH --array 1-8
```

```
# Load modules
module load IQ-TREE/2.2.2.6

# Setting sample file
SAMPFILE=/bigdata/adaskaveglab/ashan007/Scripts/Strain_CSV/IQtree2_Parsnp_CFML_Final.cs

CPU=1

if [ $SLURM_CPUS_ON_NODE ]; then
 CPU=$SLURM_CPUS_ON_NODE
fi

N=${SLURM_ARRAY_TASK_ID}

if [ ! $N ]; then
 N=$1
fi

if [ ! $N ]; then
 echo "need to provide a number by --array slurm or on the cmdline"
 exit
fi

IFS=,
tail -n +2 $SAMPFILE | sed -n ${N}p | while read RUN ALIGNMENT MODEL MFDIR IQDIR OUTGR
do
  if [ -d "$IQDIR" ]; then
    echo "Skipping $RUN - Directory $IQDIR already exists"
  else
    mkdir -p "$IQDIR"
    echo "Created directory $IQDIR and starting $RUN"

    # conditional statement for the outgroup samples
    if [[ "$RUN" == "CA_XAJ_XAP_and_NCBI_XAJ_XAP_and_Outgroup" ]]; then
      iqtree2 -s "$ALIGNMENT" -mem 32G -T AUTO -m "$MODEL" -B 1000 -o "$OUTGROUP" --pre
    else
      iqtree2 -s "$ALIGNMENT" -mem 32G -T AUTO -m "$MODEL" -B 1000 --prefix "$IQDIR/IQ
    fi
  fi
done
```

5.4.5 IQTree2_MF_Gubbins_Final.sh

```
#!/bin/bash -l

#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=32G
#SBATCH --time=14-00:00:00
##SBATCH --output=my.stdout
#SBATCH --mail-user=ashan007@ucr.edu
#SBATCH --mail-type=ALL
#SBATCH --job-name="IQtree2"
#SBATCH -p batch
#SBATCH --array 1-8


# Load modules
module load IQ-TREE/2.2.2.6

# Before running IQtree2 I removed the .fna.ref and .fna extensions from the alignment file:
# sed -i 's/\.fna\.ref\|\.fna//g' CA_XAJ.masked.aln
# sed -i 's/\.fna\.ref\|\.fna//g' CA_XAJ_and_NCBI26.masked.aln
# sed -i 's/\.fna\.ref\|\.fna//g' CA_XAJ_XAP.masked.aln
# sed -i 's/\.fna\.ref\|\.fna//g' CA_XAJ_XAP_and_NCBI_XAJ_XAP.masked.aln
# sed -i 's/\.fna\.ref\|\.fna//g' CA_XAP.masked.aln
# sed -i 's/\.fna\.ref\|\.fna//g' CA_XAP_and_NCBI22.masked.aln
# sed -i 's/\.fna\.ref\|\.fna//g' CA_XAJ_XAP_and_NCBI_XAJ_XAP_and_Outgroup.masked.aln

# Setting sample file
SAMPFILE=/bigdata/adaskaveglab/ashan007/Scripts/Strain_CSV/IQtree2_Parsnp_Gubbins_Final.csv

CPU=1

if [ $SLURM_CPUS_ON_NODE ]; then
 CPU=$SLURM_CPUS_ON_NODE
fi

N=${SLURM_ARRAY_TASK_ID}

if [ ! $N ]; then
 N=$1
fi

if [ ! $N ]; then
 echo "need to provide a number by --array slurm or on the cmdline"
 exit
```

```
fi

IFS=,
tail -n +2 $SAMPFILE | sed -n ${N}p | while read RUN ALIGNMENT MODEL MFDIRECTORY IQDIR
do
  if [ -d "$MFDIRECTORY" ]; then
    echo "Skipping $RUN - Directory $MFDIRECTORY already exists"
  fi
  if [[ ! -d "$MFDIRECTORY" ]]; then
    mkdir -p "$MFDIRECTORY"
    echo "Created directory $MFDIRECTORY and starting $RUN"
  # conditional statement for the outgroup samples
    iqtree2 -s "$ALIGNMENT" -mem 32G -T AUTO -m MFP --prefix "$MFDIRECTORY/MFout"
  fi
done
```

5.4.6 IQTree2_Gubbins_Final.sh

```
#!/bin/bash -l

#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=32G
#SBATCH --time=14-00:00:00
##SBATCH --output=my.stdout
#SBATCH --mail-user=ashan007@ucr.edu
#SBATCH --mail-type=ALL
#SBATCH --job-name="IQtree2"
#SBATCH -p batch
#SBATCH --array 1-8


# Load modules
module load IQ-TREE/2.2.2.6

# Setting sample file
SAMPFILE=/bigdata/adaskaveglab/ashan007/Scripts/Strain_CSV/IQtree2_Parsnp_Gubbins_Final
CPU=1

if [ $SLURM_CPUS_ON_NODE ]; then
 CPU=$SLURM_CPUS_ON_NODE
fi

N=${SLURM_ARRAY_TASK_ID}
```

```
if [ ! $N ]; then
 N=$1
fi

if [ ! $N ]; then
 echo "need to provide a number by --array slurm or on the cmdline"
 exit
fi

IFS=,
tail -n +2 $SAMPFILE | sed -n ${N}p | while read RUN ALIGNMENT MODEL MFDIR IQDIR OUTGROUP
do
  if [ -d "$IQDIR" ]; then
    echo "Skipping $RUN - Directory $IQDIR already exists"
  else
    mkdir -p "$IQDIR"
    echo "Created directory $IQDIR and starting $RUN"

    # conditional statement for the outgroup samples
    if [[ "$RUN" == "CA_XAJ_XAP_and_NCBI_XAJ_XAP_and_Outgroup" ]]; then
      iqtree2 -s "$ALIGNMENT" -mem 32G -T AUTO -m "$MODEL" -B 1000 -o "$OUTGROUP" --prefix "$IQDI
    else
      iqtree2 -s "$ALIGNMENT" -mem 32G -T AUTO -m "$MODEL" -B 1000 --prefix "$IQDIR/IQout"
    fi
  fi
done
```

5.4.7 CA_XAJ_hierBAPS.R

```
library(rhierbaps)
library(ape)
library(ggtree)
library(phytools)
library(ggplot2)
library(adegenet)
library(poppr)
library(RColorBrewer)
setwd("/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5_Analyses/rhierBAPS/Parsn

#-------------------------------------------------------------------------------
# RHeirBAPS
#-------------------------------------------------------------------------------
# Resource: https://github.com/gtonkinhill/rhierbaps
```

```
# Load fasta and generate matrix
Fasta = "CA_XAJ_CFML.filtered.fasta"
snp.matrix = load_fasta(Fasta)

# Running hierBAPS
# No difference in n.pops = 20
hb.results = hierBAPS(snp.matrix, max.depth = 2, n.pops = 20, quiet = TRUE, n.cores = 8

# Plotting Results
tree = "IQout.treefile"
Tree = ape::read.tree(tree)

# A simple coloured tree allows us to see the top level cluster assignment from hierBA
gg = ggtree(Tree, layout = "circular")
gg = gg %<+% hb.results$partition.df
gg = gg + geom_tippoint(aes(color = factor(`level 1`)))
gg

gg = ggtree(Tree, layout = "circular", branch.length = "none")
gg = gg %<+% hb.results$partition.df
gg = gg + geom_tippoint(aes(color = factor(`level 1`)))
gg = gg + theme(legend.position = "right")
gg = gg + geom_tiplab(aes(label = `level 2`), size = 1, offset = 1)
gg

# Plotting a specific cluster
plot_sub_cluster(hb.results, Tree, level = 1, sub.cluster = 1)

# the log marginal likelihoods given for each level
hb.results$lml.list

# save the partition file
write.csv(hb.results$partition.df, file = file.path("Outputs/CA_XAJ_115_hierbaps_parti
          col.names = TRUE, row.names = FALSE)

#------------------------------------------------------------------------------
# DAPC
#------------------------------------------------------------------------------

# Import the RHeirBAPS clusters to serve as the populations
# Importing the clusters from the hierBAPS run to set as the pop
HB_Clust= read.csv("Outputs/CA_XAJ_115_hierbaps_partition_npop20.csv", header =T)
HB_Clust = as.data.frame(HB_Clust)

# Import Fasta alignment for use in Adegenet
```

```
MSA = fasta2DNAbin(Fasta)
# Convert to matrix
MSA_Matrix = as.matrix(MSA)
# Convert to genind
MSA_Genind <- DNAbin2genind(MSA_Matrix)
MSA_Genind
# Set factors
MSA_Genind$pop <- as.factor(HB_Clust$level.1)
MSA_Genind$other <- as.list(HB_Clust$Isolate)
# Setting colors
cols <- brewer.pal(n = nPop(MSA_Genind), name = "Paired")

# identify number of PCs to use for n.pca
Xval <- xvalDapc(tab(MSA_Genind, NA.method = "mean"), pop(MSA_Genind), parallel = "multicore", n.
Xval
# Generating DAPC
DAPC <- dapc(MSA_Genind, pop=as.factor(unlist(MSA_Genind$pop)),
             n.pca=25, n.da=4, var.loadings=T, pca.info=T)
group_levels <- levels(DAPC$grp)

# DAPC Scatterplot
scatter.dapc(DAPC, grp=as.factor(unlist(MSA_Genind$pop)), legend=T,
             col=cols, cex.lab=1.5, clabel = 0, cleg = 0.75,
             cellipse = 10, ratio.da=.15, posi.leg=("topleft"),
             posi.da = "topright")
quartz.save("Outputs/CA_XAJ_115_DAPC.pdf", type="pdf")
```

5.4.8 CA_XAJ_and_NCBI24_hierBAPS.R

```
library(rhierbaps)
library(ape)
library(ggtree)
library(phytools)
library(ggplot2)
library(adegenet)
library(poppr)
library(RColorBrewer)
setwd("/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5_Analyses/rhierBAPS/Parsn

#-------------------------------------------------------------------------------
# RHeirBAPS
#-------------------------------------------------------------------------------
# Resource: https://github.com/gtonkinhill/rhierbaps
```

```
# Load fasta and generate matrix
Fasta = "CA_XAJ_and_NCBI24_CFML.filtered.fasta"
snp.matrix = load_fasta(Fasta)

# Running hierBAPS
# No difference in n.pops = 20
hb.results = hierBAPS(snp.matrix, max.depth = 2, n.pops = 20, quiet = TRUE, n.cores = 8

# Plotting Results
tree = "IQout.treefile"
Tree = ape::read.tree(tree)

# A simple coloured tree allows us to see the top level cluster assignment from hierBA
gg = ggtree(Tree, layout = "circular")
gg = gg %<+% hb.results$partition.df
gg = gg + geom_tippoint(aes(color = factor(`level 1`)))
gg

gg = ggtree(Tree, layout = "circular", branch.length = "none")
gg = gg %<+% hb.results$partition.df
gg = gg + geom_tippoint(aes(color = factor(`level 1`)))
gg = gg + theme(legend.position = "right")
gg = gg + geom_tiplab(aes(label = `level 2`), size = 1, offset = 1)
gg

# Plotting a specific cluster
plot_sub_cluster(hb.results, Tree, level = 1, sub.cluster = 1)

# the log marginal likelihoods given for each level
hb.results$lml.list

# save the partition file
write.csv(hb.results$partition.df, file = file.path("Outputs/CA_XAJ_115_and_NCBI24_hier
          col.names = TRUE, row.names = FALSE)

#------------------------------------------------------------------------------
# DAPC
#------------------------------------------------------------------------------

# Import the RHeirBAPS clusters to serve as the populations
# Importing the clusters from the hierBAPS run to set as the pop
HB_Clust= read.csv("Outputs/CA_XAJ_115_and_NCBI24_hierbaps_partition_npop20.csv", heade
HB_Clust = as.data.frame(HB_Clust)

# Import Fasta alignment for use in Adegenet
```

```
MSA = fasta2DNAbin(Fasta)
# Convert to matrix
MSA_Matrix = as.matrix(MSA)
# Convert to genind
MSA_Genind <- DNAbin2genind(MSA_Matrix)
MSA_Genind
# Set factors
MSA_Genind$pop <- as.factor(HB_Clust$level.1)
MSA_Genind$other <- as.list(HB_Clust$Isolate)
# Setting colors
cols <- brewer.pal(n = nPop(MSA_Genind), name = "Paired")

# identify number of PCs to use for n.pca
Xval <- xvalDapc(tab(MSA_Genind, NA.method = "mean"), pop(MSA_Genind), parallel = "multicore", n.
Xval
# Generating DAPC
DAPC <- dapc(MSA_Genind, pop=as.factor(unlist(MSA_Genind$pop)),
             n.pca=40, n.da=7, var.loadings=T, pca.info=T)
group_levels <- levels(DAPC$grp)

# DAPC Scatterplot
scatter.dapc(DAPC, grp=as.factor(unlist(MSA_Genind$pop)), legend=T,
             col=cols, cex.lab=1.5, clabel = 0, cleg = 0.75,
             cellipse = 15, ratio.da=.15, posi.leg=("topleft"),
             posi.da = "topright")
quartz.save("Outputs/CA_XAJ_115_and_NCBI24_DAPC.pdf", type="pdf")
```

5.4.9 CA_XAJ_XAP_and_NCBI_XAJ_XAP_and_Outgroup_hierBAPS.R

```
library(rhierbaps)
library(ape)
library(ggtree)
library(phytools)
library(ggplot2)
library(adegenet)
library(poppr)
library(RColorBrewer)
setwd("/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5_Analyses/rhierBAPS/Parsn

#-------------------------------------------------------------------------------
# RHeirBAPS
#-------------------------------------------------------------------------------
# Resource: https://github.com/gtonkinhill/rhierbaps
```

```
# Load fasta and generate matrix
Fasta = "CA_XAJ_XAP_and_NCBI_XAJ_XAP_and_Outgroup_CFML.filtered.fasta"
snp.matrix = load_fasta(Fasta)

# Running hierBAPS
# No difference in n.pops = 20
hb.results = hierBAPS(snp.matrix, max.depth = 2, n.pops = 20, quiet = TRUE, n.cores = 8

# Plotting Results
tree = "IQout.treefile"
Tree = ape::read.tree(tree)

# A simple coloured tree allows us to see the top level cluster assignment from hierBA
gg = ggtree(Tree, layout = "circular")
gg = gg %<+% hb.results$partition.df
gg = gg + geom_tippoint(aes(color = factor(`level 1`)))
gg

gg = ggtree(Tree, layout = "circular", branch.length = "none")
gg = gg %<+% hb.results$partition.df
gg = gg + geom_tippoint(aes(color = factor(`level 1`)))
gg = gg + theme(legend.position = "right")
gg = gg + geom_tiplab(aes(label = `level 2`), size = 1, offset = 1)
gg

# Plotting a specific cluster
plot_sub_cluster(hb.results, Tree, level = 1, sub.cluster = 1)

# the log marginal likelihoods given for each level
hb.results$lml.list

# save the partition file
write.csv(hb.results$partition.df, file = file.path("Outputs/CA_XAJ_115_XAP_and_NCBI_XA
          col.names = TRUE, row.names = FALSE)

#------------------------------------------------------------------------------
# DAPC
#------------------------------------------------------------------------------

# Import the RHeirBAPS clusters to serve as the populations
# Importing the clusters from the hierBAPS run to set as the pop
HB_Clust= read.csv("Outputs/CA_XAJ_115_XAP_and_NCBI_XAJ_XAP_and_Outgroup_hierbaps_parti
HB_Clust = as.data.frame(HB_Clust)

# Import Fasta alignment for use in Adegenet
```

```
MSA = fasta2DNAbin(Fasta)
# Convert to matrix
MSA_Matrix = as.matrix(MSA)
# Convert to genind
MSA_Genind <- DNAbin2genind(MSA_Matrix)
MSA_Genind
# Set factors
MSA_Genind$pop <- as.factor(HB_Clust$level.1)
MSA_Genind$other <- as.list(HB_Clust$Isolate)
# Setting colors
cols <- brewer.pal(n = nPop(MSA_Genind), name = "Paired")

# identify number of PCs to use for n.pca
Xval <- xvalDapc(tab(MSA_Genind, NA.method = "mean"), pop(MSA_Genind), parallel = "multicore", n.
Xval
# Generating DAPC
DAPC <- dapc(MSA_Genind, pop=as.factor(unlist(MSA_Genind$pop)),
             n.pca=20, n.da=2, var.loadings=T, pca.info=T)
group_levels <- levels(DAPC$grp)

# DAPC Scatterplot
scatter.dapc(DAPC, grp=as.factor(unlist(MSA_Genind$pop)), legend=T,
             col=cols, cex.lab=1.5, clabel = 0, cleg = 0.75,
             cellipse = 15, ratio.da=.15, posi.leg=("topleft"),
             posi.da = "topright")
quartz.save("Outputs/CA_XAJ_115_XAP_and_NCBI_XAJ_XAP_and_Outgroup_DAPC.pdf", type="pdf")
```

5.5 Molecular Clock

5.5.1 CA_XAJ_CFML_BactDating.R

```
# Aidan Shands
# 09.13.2024

# install BactBating
#devtools::install_github("xavierdidelot/BactDating")
library(BactDating)
library(ape)
library(dplyr)
library(lubridate)
library(coda)
library(treeio)
library(ggtree)
```

```
# Resources:
# https://xavierdidelot.github.io/BactDating/articles/yourData.html
# https://githubhelp.com/xavierdidelot/BactDating

# set wd
setwd('/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5_Analyses/Bactl
# import dates
Strain_Dates = read.csv("CA_XAJ_115_and_NCBI24_Dates.csv")


# I will first try this using the CFML corrected CA_XAJ run.
t=loadCFML("/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5_Analyses/
t
Tips=t$tip.label
Tips
#write.csv(Tips, "tips_check.csv")
# ensure that the Tips have the correct date since the Dates.csv
# are not in the same order as the tree tip labels (Tips)
ordered_dates <- Strain_Dates %>%
  slice(match(Tips, ID))
ordered_dates
Dates = ordered_dates$Year

#
sum(t$edge.length) # >1

# if your tree is unrooted you can find an optimal root using
rooted=initRoot(t,Dates)

# Before running the main bactdate analysis, it is also a good idea to assess
# the strength of the temporal signal using a root-to-tip linear regression:
r=roottotip(rooted,Dates)

# main command

# BactDating is a MCMC algorithm which needs to be run for long enough to ensure
# that the results are meaningful. The length of the algorithm can be controlled
# using the nbIts argument of the bactdate function, which by default is 10e4 but
# may need increasing for example ten folds:

# using the re-rooted tree from initRoot() with relaxed clock
res=bactdate(rooted, Dates, showProgress = T, nbIts=1e6, model='relaxedgamma')
# using the re-rooted tree from initRoot() with relaxed clock qith 1e8
res2=bactdate(rooted, Dates, showProgress = T, nbIts=1e8, model='relaxedgamma')
```

```
res3=bactdate(rooted, Dates, showProgress = T, nbIts=1e6)
# use the unrooted tree
res4=bactdate(t,Dates, nbIts=1e6, useRec = T, showProgress = T)
# To test the MCMC convergence, a good start is to check that the traces of the
# parameters look stable using the command:
plot(res,'trace')
# Credible Intervals
plot(res,'treeCI')

plot(res,'treeRoot')
# export tree with the CI data
# Reference https://github.com/xavierdidelot/BactDating/issues/9
Tree2 = as.treedata.resBactDating(res)
obj=methods::new('treedata',phylo=Tree2[[1]],data=dplyr::as_tibble(as.data.frame(Tree2[[2]])))

ggtree(obj) + geom_range(range='length_0.95_HPD', color='red', alpha=.6, size=2)
write.beast(obj, file = "CA_XAP_CFML_BDout_withCI_Rooted_Relaxedd_1e8MCMC_CLT.nwk")
```

5.5.2 CA_XAJ_Gub_BactDating.R

```
# Aidan Shands
# 09.13.2024

# install BactBating
#devtools::install_github("xavierdidelot/BactDating")
library(BactDating)
library(ape)
library(dplyr)
library(lubridate)
library(coda)
library(treeio)
library(ggtree)


# Resources:
# https://xavierdidelot.github.io/BactDating/articles/yourData.html
# https://githubhelp.com/xavierdidelot/BactDating

# set wd
setwd('/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5_Analyses/BactDating/CA_X
# import dates
Strain_Dates = read.csv("CA_XAJ_Dates.csv")
```

```
# I will first try this using the CFML corrected CA_XAJ run.
t=loadGubbins("/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5_Analys
t
Tips=t$tip.label
Tips
#write.csv(Tips, "tips_check.csv")
# ensure that the Tips have the correct date since the Dates.csv
# are not in the same order as the tree tip labels (Tips)
ordered_dates <- Strain_Dates %>%
  slice(match(Tips, ID))
ordered_dates
Dates = ordered_dates$Year

#
sum(t$edge.length) # >1

# if your tree is unrooted you can find an optimal root using
rooted=initRoot(t,Dates)

# Before running the main bactdate analysis, it is also a good idea to assess
# the strength of the temporal signal using a root-to-tip linear regression:
r=roottotip(rooted,Dates)

# main command

# BactDating is a MCMC algorithm which needs to be run for long enough to ensure
# that the results are meaningful. The length of the algorithm can be controlled
# using the nbIts argument of the bactdate function, which by default is 10e4 but
# may need increasing for example ten folds:

# using the re-rooted tree from initRoot() with relaxed clock
res=bactdate(rooted, Dates, showProgress = T, nbIts=1e6, model='relaxedgamma')
# using the re-rooted tree from initRoot() with relaxed clock qith 1e8
res2=bactdate(rooted, Dates, showProgress = T, nbIts=1e8, model='relaxedgamma')

res3=bactdate(rooted, Dates, showProgress = T, nbIts=1e6)
# use the unrooted tree
res4=bactdate(t,Dates, nbIts=1e6, useRec = T, showProgress = T)
# To test the MCMC convergence, a good start is to check that the traces of the
# parameters look stable using the command:
plot(res,'trace')
# Credible Intervals
plot(res,'treeCI')

plot(res,'treeRoot')
```

```
# For further testing of convergence, you can export the BactDating result to the format required
mcmc=as.mcmc(res)

#You can then compute for example the effective sample size of the parameters which should be at
effectiveSize(mcmc)
# export tree with the CI data
# Reference https://github.com/xavierdidelot/BactDating/issues/9
Tree2 = as.treedata.resBactDating(res)
obj=methods::new('treedata',phylo=Tree2[[1]],data=dplyr::as_tibble(as.data.frame(Tree2[[2]])))

ggtree(obj) + geom_range(range='length_0.95_HPD', color='red', alpha=.6, size=2)
write.beast(obj, file = "CA_XAP_CFML_BDout_withCI_Rooted_Relaxedd_1e8MCMC_CLT.nwk")

resX=bactdate(rooted,rep(2015,length(Dates)),  showProgress = T)
modelcompare(res,resX)
```

5.5.3 CA_XAJ_CFML_IQtree_BactDating.R

```
# Aidan Shands
# 09.13.2024

# install BactBating
#devtools::install_github("xavierdidelot/BactDating")
library(BactDating)
library(ape)
library(dplyr)
library(lubridate)
library(coda)
library(treeio)
library(ggtree)


# Resources:
# https://xavierdidelot.github.io/BactDating/articles/yourData.html
# https://githubhelp.com/xavierdidelot/BactDating

# set wd
setwd('/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5_Analyses/BactDating/CA_X
# import dates
Strain_Dates = read.csv("CA_XAJ_Dates.csv")


# I will first try this using the CFML corrected CA_XAJ run.
t=loadGubbins("/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5_Analyses/Gubbins
```

```
t
Tips=t$tip.label
Tips
#write.csv(Tips, "tips_check.csv")
# ensure that the Tips have the correct date since the Dates.csv
# are not in the same order as the tree tip labels (Tips)
ordered_dates <- Strain_Dates %>%
  slice(match(Tips, ID))
ordered_dates
Dates = ordered_dates$Year

#
sum(t$edge.length) # >1

# if your tree is unrooted you can find an optimal root using
rooted=initRoot(t,Dates)

# Before running the main bactdate analysis, it is also a good idea to assess
# the strength of the temporal signal using a root-to-tip linear regression:
r=roottotip(rooted,Dates)

# main command

# BactDating is a MCMC algorithm which needs to be run for long enough to ensure
# that the results are meaningful. The length of the algorithm can be controlled
# using the nbIts argument of the bactdate function, which by default is 10e4 but
# may need increasing for example ten folds:

# using the re-rooted tree from initRoot() with relaxed clock
res=bactdate(rooted, Dates, showProgress = T, nbIts=1e6, model='relaxedgamma')
# using the re-rooted tree from initRoot() with relaxed clock qith 1e8
res2=bactdate(rooted, Dates, showProgress = T, nbIts=1e8, model='relaxedgamma')

res3=bactdate(rooted, Dates, showProgress = T, nbIts=1e6)
# use the unrooted tree
res4=bactdate(t,Dates, nbIts=1e6, useRec = T, showProgress = T)
# To test the MCMC convergence, a good start is to check that the traces of the
# parameters look stable using the command:
plot(res,'trace')
# Credible Intervals
plot(res,'treeCI')

plot(res,'treeRoot')

# For further testing of convergence, you can export the BactDating result to the form
```

```
mcmc=as.mcmc(res)

#You can then compute for example the effective sample size of the parameters which should be at
effectiveSize(mcmc)
# export tree with the CI data
# Reference https://github.com/xavierdidelot/BactDating/issues/9
Tree2 = as.treedata.resBactDating(res)
obj=methods::new('treedata',phylo=Tree2[[1]],data=dplyr::as_tibble(as.data.frame(Tree2[[2]])))

ggtree(obj) + geom_range(range='length_0.95_HPD', color='red', alpha=.6, size=2)
write.beast(obj, file = "CA_XAP_CFML_BDout_withCI_Rooted_Relaxedd_1e8MCMC_CLT.nwk")

resX=bactdate(rooted,rep(2015,length(Dates)),  showProgress = T)
modelcompare(res,resX)
```

5.5.4 CA_XAJ_Gub_IQtree_BactDating.R

```
# Aidan Shands
# 09.13.2024

# install BactBating
#devtools::install_github("xavierdidelot/BactDating")
library(BactDating)
library(ape)
library(dplyr)
library(lubridate)
library(coda)
library(treeio)
library(ggtree)


# Resources:
# https://xavierdidelot.github.io/BactDating/articles/yourData.html
# https://githubhelp.com/xavierdidelot/BactDating

# set wd
setwd('/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5_Analyses/BactDating/CA_X
# import dates
Strain_Dates = read.csv("CA_XAJ_Dates.csv")


# I will first try this using the CFML corrected CA_XAJ run.
t=read.tree("IQout.treefile")
t
```

```
Tips=t$tip.label
Tips
#write.csv(Tips, "tips_check.csv")
# ensure that the Tips have the correct date since the Dates.csv
# are not in the same order as the tree tip labels (Tips)
ordered_dates <- Strain_Dates %>%
  slice(match(Tips, ID))
ordered_dates
Dates = ordered_dates$Year

#
sum(t$edge.length) # <1
# L = 4222345 which is the number of sites used to build the tree (gathered from IQout
L=4222345
t$edge.length=t$edge.length*L
# if your tree is unrooted you can find an optimal root using
rooted=initRoot(t,Dates)

# Before running the main bactdate analysis, it is also a good idea to assess
# the strength of the temporal signal using a root-to-tip linear regression:
r=roottotip(rooted,Dates)

# main command

# BactDating is a MCMC algorithm which needs to be run for long enough to ensure
# that the results are meaningful. The length of the algorithm can be controlled
# using the nbIts argument of the bactdate function, which by default is 10e4 but
# may need increasing for example ten folds:

# using the re-rooted tree from initRoot() with relaxed clock
res=bactdate(rooted, Dates, showProgress = T, nbIts=1e6, model='relaxedgamma')
# using the re-rooted tree from initRoot() with relaxed clock qith 1e8
res2=bactdate(rooted, Dates, showProgress = T, nbIts=1e8, model='relaxedgamma')

res3=bactdate(rooted, Dates, showProgress = T, nbIts=1e6)
# use the unrooted tree
res4=bactdate(t,Dates, nbIts=1e6, useRec = T, showProgress = T)
# To test the MCMC convergence, a good start is to check that the traces of the
# parameters look stable using the command:
plot(res,'trace')
# Credible Intervals
plot(res,'treeCI')

plot(res,'treeRoot')
```

```
# For further testing of convergence, you can export the BactDating result to the format required
mcmc=as.mcmc(res)

#You can then compute for example the effective sample size of the parameters which should be at
effectiveSize(mcmc)
# export tree with the CI data
# Reference https://github.com/xavierdidelot/BactDating/issues/9
Tree2 = as.treedata.resBactDating(res)
obj=methods::new('treedata',phylo=Tree2[[1]],data=dplyr::as_tibble(as.data.frame(Tree2[[2]])))

ggtree(obj) + geom_range(range='length_0.95_HPD', color='red', alpha=.6, size=2)
write.beast(obj, file = "CA_XAP_CFML_BDout_withCI_Rooted_Relaxedd_1e8MCMC_CLT.nwk")

resX=bactdate(rooted,rep(2015,length(Dates)),  showProgress = T)
modelcompare(res,resX)
```

5.6 Type III Effectors

5.6.1 Xaj_Xap_T3SSE_tBLASTn.sh

```
#!/bin/bash -l

#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=32G
#SBATCH --time=20-00:00:00
##SBATCH --output=my.stdout
#SBATCH --mail-user=ashan007@ucr.edu
#SBATCH --mail-type=ALL
#SBATCH --job-name="BLAST"
#SBATCH -p batch
#SBATCH --array 1-140


# Load modules
module load ncbi-blast/2.16.0+

# Setting Variables
SAMPFILE=/bigdata/adaskaveglab/ashan007/Scripts/Strain_CSV/Strains.csv
BLASTDIR=/bigdata/adaskaveglab/ashan007/Assembly_Workflow_PlatanusB/Step5_Analyses/Effectors/tBLA
T3SSE=/bigdata/adaskaveglab/ashan007/Effectors/Ref_Xan_T3SSEs/Xanthomonas_T3_effectors.AA.fasta
GENOMEDIR=/bigdata/adaskaveglab/ashan007/Assembly_Workflow_PlatanusB/Step5_Analyses/Effectors/Res
```

```
CPU=1

if [ $SLURM_CPUS_ON_NODE ]; then
 CPU=$SLURM_CPUS_ON_NODE
fi

N=${SLURM_ARRAY_TASK_ID}

if [ ! $N ]; then
 N=$1
fi

if [ ! $N ]; then
 echo "need to provide a number by --array slurm or on the cmdline"
 exit
fi

IFS=,
tail -n +2 $SAMPFILE | sed -n ${N}p | while read SID STRAIN READ1 READ2 ORGANISM
do
  if [ -d $BLASTDIR/$STRAIN ]; then
    echo "Skipping $STRAIN - Directory already exists"
  fi
  if [[ ! -d $BLASTDIR/$STRAIN ]]; then
    mkdir $BLASTDIR/$STRAIN

    makeblastdb -in $GENOMEDIR/$STRAIN.fna -input_type fasta -dbtype nucl

    tblastn -query $T3SSE \
    -db $GENOMEDIR/$STRAIN.fna \
    -out $BLASTDIR/$STRAIN/$STRAIN.output \
    -evalue 1e-10 \
    -num_threads 8 \
    -max_target_seqs 5 \
    -max_hsps 5

    tblastn -query $T3SSE \
    -db $GENOMEDIR/$STRAIN.fna \
    -out $BLASTDIR/$STRAIN/$STRAIN.txt \
    -evalue 1e-10 \
    -num_threads 8 \
    -max_target_seqs 5 \
    -max_hsps 5 \
    -outfmt "6 qseqid sseqid pident length mismatch gapopen qstart qend sstart send eva
  fi
```

```
done
```

5.6.2 Xaj_Xap_T3SSE_BLASTp.sh

```
#!/bin/bash -l

#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=32G
#SBATCH --time=5-00:00:00
##SBATCH --output=my.stdout
#SBATCH --mail-user=ashan007@ucr.edu
#SBATCH --mail-type=ALL
#SBATCH --job-name="BLASTp"
#SBATCH -p batch
#SBATCH --array 1-140


# Load modules
module load ncbi-blast/2.16.0+

# Setting Variables
SAMPFILE=/bigdata/adaskaveglab/ashan007/Scripts/Strain_CSV/Strains.csv
BLASTDIR=/bigdata/adaskaveglab/ashan007/Assembly_Workflow_PlatanusB/Step5_Analyses/Effectors/BLAS
T3SSE=/bigdata/adaskaveglab/ashan007/Assembly_Workflow_PlatanusB/Step5_Analyses/Effectors/Resourc
PROTDIR=/bigdata/adaskaveglab/ashan007/Assembly_Workflow_PlatanusB/Step5_Analyses/Effectors/Resou

# before running this script I ran the following commands
# srun -p short -c 4 --mem=10g --pty bash -l
# module load ncbi-blast/2.16.0+
# makeblastdb -in Xanthomonas_T3_effectors.AA.fasta -input_type fasta -dbtype prot

CPU=1

if [ $SLURM_CPUS_ON_NODE ]; then
 CPU=$SLURM_CPUS_ON_NODE
fi

N=${SLURM_ARRAY_TASK_ID}

if [ ! $N ]; then
 N=$1
fi
```

```
if [ ! $N ]; then
 echo "need to provide a number by --array slurm or on the cmdline"
 exit
fi

IFS=,
tail -n +2 $SAMPFILE | sed -n ${N}p | while read SID STRAIN READ1 READ2 ORGANISM
do
  if [ -d $BLASTDIR/$STRAIN ]; then
    echo "Skipping $STRAIN - Directory already exists"
  fi
  if [[ ! -d $BLASTDIR/$STRAIN ]]; then
    mkdir $BLASTDIR/$STRAIN

    blastp -query $PROTDIR/$STRAIN.faa \
    -db $T3SSE \
    -out $BLASTDIR/$STRAIN/$STRAIN.output \
    -evalue 1e-10 \
    -num_threads 8 \
    -max_target_seqs 5 \
    -max_hsps 5

    blastp -query $PROTDIR/$STRAIN.faa \
    -db $T3SSE \
    -out $BLASTDIR/$STRAIN/$STRAIN.txt \
    -evalue 1e-10 \
    -num_threads 8 \
    -max_target_seqs 5 \
    -max_hsps 5 \
    -outfmt "6 qseqid sseqid pident length mismatch gapopen qstart qend sstart send eva
  fi
done
```

5.6.3 Filter_tBLASTn.py

```
import os
import pandas as pd

# Define the root directory containing all the subdirectories with Sample.txt files
root_dir = '/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5_Analyses/

# Define the column headers
columns = ['qseqid', 'sseqid', 'pident', 'length', 'mismatch', 'gapopen',
           'qstart', 'qend', 'sstart', 'send', 'evalue', 'bitscore', 'qlen', 'slen']
```

```
# Loop through each subdirectory in the root directory
for subdir in os.listdir(root_dir):
    subdir_path = os.path.join(root_dir, subdir)

    # Check if it's a directory
    if os.path.isdir(subdir_path):
        sample_file_path = os.path.join(subdir_path, f'{subdir}.txt')

        # Check if the Sample.txt file exists in the subdirectory
        if os.path.isfile(sample_file_path):
            # Read the tab-delimited file without headers
            df = pd.read_csv(sample_file_path, sep='\t', header=None, names=columns)

            # Apply the first filter: pident > 80
            filter1 = df['pident'] > 80

            # Apply the second filter: (length/qlen) * 100 > 80
            filter2 = (df['length'] / df['qlen']) * 100 > 80

            # Apply both filters
            filtered_df = df[filter1 & filter2]

            # Define the output file path
            output_file_path = os.path.join(subdir_path, f'{subdir}.tBLASTn.filtered.txt')

            # Write the filtered data to a new file with headers
            filtered_df.to_csv(output_file_path, sep='\t', index=False, header=True)

print("Filtering complete for all samples.")
```

5.6.4 Filter_tBLASTn.RemoveDups.py

```
import os
import pandas as pd

# Directories
# Method 1
#blastp_dir = '/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5_Analyses/Effecto
#output_dir = '/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5_Analyses/Effecto
# Method 2
blastp_dir = '/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5_Analyses/Effector
output_dir = '/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5_Analyses/Effector

# Loop through all .csv files in the directory
for file_name in os.listdir(blastp_dir):
```

```
    if file_name.endswith('.txt'):  # Process only .txt files
        input_file = os.path.join(blastp_dir, file_name)

        # Extract the basename without the extension
        base_name = os.path.splitext(file_name)[0]
        output_file = os.path.join(output_dir, f"{base_name}.filtered.csv")

        try:
            # Read the .csv file into a DataFrame
            df = pd.read_csv(input_file, encoding='utf-8', sep='\t')

            # Strip whitespace from column names (if any)
            df.columns = df.columns.str.strip()

            # Filter rows: Keep the one with the highest pident for each qseqid
            qseqid_filtered = df.loc[df.groupby('qseqid')['pident'].idxmax()]

            # Save the filtered DataFrame to a new file
            qseqid_filtered.to_csv(output_file, index=False)  # Save as tab-separated
            print(f"Filtered file saved: {output_file}")

        except Exception as e:
            print(f"Error processing {file_name}: {e}")
```

5.6.5 BLAST_Matrix_FamilyName.method1.py

```
import os
import pandas as pd

# File paths
t3e_guide_file = '/Volumes/AS_Postdoc/Postdoc2024/T3SS_Effectors/T3SS_Xanthomonas_Refe
compare_dir = '/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5_Analys
output_matrix_file = '/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5

# Step 1: Read T3E_Guide.csv to create a dictionary mapping RefSeq_ID to Family
t3e_guide_df = pd.read_csv(t3e_guide_file)
refseq_to_family = dict(zip(t3e_guide_df['RefSeq_ID'], t3e_guide_df['Family']))

# Step 2: Get a list of all effector families (for matrix columns)
families = sorted(t3e_guide_df['Family'].unique())

# Step 3: Initialize an empty matrix (list of dicts) where each sample will be a row
matrix = []
```

```
# Step 4: Loop through each sample's .T3E.csv file in the Compare_BLASTp_and_tBLASTn directory
for file in os.listdir(compare_dir):
    if file.endswith('.csv') and not file.startswith('._'):
        sample = file.split('.')[0]  # Extract sample name from the file
        file_path = os.path.join(compare_dir, file)

        # Read the .T3E.csv file
        t3e_df = pd.read_csv(file_path)

        # Step 5: Initialize a dictionary to hold presence/absence for this sample
        sample_data = {'Strain': sample}

        # Check presence of each family based on qseqid in t3e_df and RefSeq_ID in T3E_Guide
        for family in families:
            # Get all RefSeq_IDs corresponding to this family
            family_refseq_ids = [refseq for refseq, fam in refseq_to_family.items() if fam == fam

            # Check if any qseqid in t3e_df is in the list of RefSeq_IDs for this family
            if any(t3e_df['qseqid'].isin(family_refseq_ids)):
                sample_data[family] = 'Present'
            else:
                sample_data[family] = 'Absent'

        # Add the sample's data to the matrix
        matrix.append(sample_data)

# Step 6: Convert the matrix (list of dicts) to a DataFrame and write to CSV
matrix_df = pd.DataFrame(matrix)

# Step 7: Drop columns where all rows have the value 'Absent'
# This works by selecting columns that have any value other than 'Absent'
matrix_df = matrix_df.loc[:, (matrix_df != 'Absent').any(axis=0)]

# Write the matrix to a CSV file
matrix_df.to_csv(output_matrix_file, index=False)

print(f'Matrix written to {output_matrix_file}')
```

5.6.6 BLAST_Matrix_RefSeqID.method1.py

```
import os
import pandas as pd

# File paths
```

```
t3e_guide_file = '/Volumes/AS_Postdoc/Postdoc2024/T3SS_Effectors/T3SS_Xanthomonas_Refer
compare_dir = '/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5_Analys
output_matrix_file = '/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5

# Step 1: Read T3E_Guide.csv to get the RefSeq_IDs (these will be the columns)
t3e_guide_df = pd.read_csv(t3e_guide_file)
refseq_ids = sorted(t3e_guide_df['RefSeq_ID'].unique())  # Use RefSeq_ID for columns

# Step 2: Initialize an empty matrix (list of dicts) where each sample will be a row
matrix = []

# Step 3: Loop through each sample's .T3E.csv file in the Compare_BLASTp_and_tBLASTn d
for file in os.listdir(compare_dir):
    if file.endswith('.csv') and not file.startswith('._'):
        sample = file.split('.')[0]  # Extract sample name from the file
        file_path = os.path.join(compare_dir, file)

        # Read the .T3E.csv file
        t3e_df = pd.read_csv(file_path, encoding='utf-8')

        # Check if expected columns are present
        if 'qseqid' not in t3e_df.columns or 'sseqid' not in t3e_df.columns:
            print(f"Warning: Missing expected columns in {file_path}. Skipping file.")
            continue

        # Step 4: Initialize a dictionary to hold presence/absence for this sample
        sample_data = {'Strain': sample}

        # Check presence of each RefSeq_ID based on sseqid in t3e_df
        for refseq_id in refseq_ids:
            # Check if sseqid in t3e_df matches the current refseq_id
            if any(t3e_df['qseqid'] == refseq_id):
                sample_data[refseq_id] = 'Present'
            else:
                sample_data[refseq_id] = 'Absent'

        # Add the sample's data to the matrix
        matrix.append(sample_data)

# Step 5: Convert the matrix (list of dicts) to a DataFrame
matrix_df = pd.DataFrame(matrix)

# Step 6: Drop columns where all rows have the value 'Absent'
matrix_df = matrix_df.loc[:, (matrix_df != 'Absent').any(axis=0)]
```

```
# Step 7: Write the matrix to a CSV file
matrix_df.to_csv(output_matrix_file, index=False)

print(f'Matrix written to {output_matrix_file} with RefSeq_ID columns and empty columns removed.'
```

5.6.7 Filter_tBLASTn.v2.py

```
import os
import pandas as pd
import numpy as np

# Define the root directory containing all the subdirectories with Sample.txt files
root_dir = '/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5_Analyses/Effectors/

# Define the column headers
columns = ['qseqid', 'sseqid', 'pident', 'length', 'mismatch', 'gapopen',
           'qstart', 'qend', 'sstart', 'send', 'evalue', 'bitscore', 'qlen', 'slen']

# Loop through each subdirectory in the root directory
for subdir in os.listdir(root_dir):
    subdir_path = os.path.join(root_dir, subdir)

    # Check if it's a directory
    if os.path.isdir(subdir_path):
        sample_file_path = os.path.join(subdir_path, f'{subdir}.txt')

        # Check if the Sample.txt file exists in the subdirectory
        if os.path.isfile(sample_file_path):
            # Read the tab-delimited file without headers
            df = pd.read_csv(sample_file_path, sep='\t', header=None, names=columns)

            # Apply the first filter: pident >= 40
            filter1 = df['pident'] >= 40

            # Apply the second filter: (length/qlen) * 100 >= 80
            filter2 = (df['length'] / df['qlen']) * 100 >= 75

            # Apply both filters
            filtered_df = df[filter1 & filter2]

            # Define the output file path
            output_file_path = os.path.join(subdir_path, f'{subdir}.BLASTp.filtered.v2.txt')

            # Write the filtered data to a new file with headers
            filtered_df.to_csv(output_file_path, sep='\t', index=False, header=True)
```

```
print("Filtering complete for all samples.")
import os
import pandas as pd
import numpy as np

# Define the root directory containing all the subdirectories with Sample.txt files
root_dir = '/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5_Analyses/

# Define the column headers
columns = ['qseqid', 'sseqid', 'pident', 'length', 'mismatch', 'gapopen',
           'qstart', 'qend', 'sstart', 'send', 'evalue', 'bitscore', 'qlen', 'slen']

# Loop through each subdirectory in the root directory
for subdir in os.listdir(root_dir):
    subdir_path = os.path.join(root_dir, subdir)

    # Check if it's a directory
    if os.path.isdir(subdir_path):
        sample_file_path = os.path.join(subdir_path, f'{subdir}.txt')

        # Check if the Sample.txt file exists in the subdirectory
        if os.path.isfile(sample_file_path):
            # Read the tab-delimited file without headers
            df = pd.read_csv(sample_file_path, sep='\t', header=None, names=columns)

            # Apply the first filter: pident >= 40
            filter1 = df['pident'] >= 40

            # Apply the second filter: (length/qlen) * 100 >= 80
            filter2 = (df['length'] / df['qlen']) * 100 >= 75

            # Apply both filters
            filtered_df = df[filter1 & filter2]

            # Define the output file path
            output_file_path = os.path.join(subdir_path, f'{subdir}.BLASTp.filtered.v2

            # Write the filtered data to a new file with headers
            filtered_df.to_csv(output_file_path, sep='\t', index=False, header=True)

print("Filtering complete for all samples.")
```

5.6.8 BLAST_Matrix_FamilyName.method2.py

```
import os
import pandas as pd

# File paths
t3e_guide_file = '/Volumes/AS_Postdoc/Postdoc2024/T3SS_Effectors/T3SS_Xanthomonas_Reference_Effec
compare_dir = '/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5_Analyses/Effecto
output_matrix_file = '/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5_Analyses/

# Step 1: Read T3E_Guide.csv to create a dictionary mapping RefSeq_ID to Family
t3e_guide_df = pd.read_csv(t3e_guide_file)
refseq_to_family = dict(zip(t3e_guide_df['RefSeq_ID'], t3e_guide_df['Family']))

# Step 2: Get a list of all effector families (for matrix columns)
families = sorted(t3e_guide_df['Family'].unique())

# Step 3: Initialize an empty matrix (list of dicts) where each sample will be a row
matrix = []

# Step 4: Loop through each sample's .T3E.csv file in the Compare_BLASTp_and_tBLASTn directory
for file in os.listdir(compare_dir):
    if file.endswith('.csv') and not file.startswith('._'):
        sample = file.split('.')[0]  # Extract sample name from the file
        file_path = os.path.join(compare_dir, file)

        # Read the .T3E.csv file
        t3e_df = pd.read_csv(file_path)

        # Step 5: Initialize a dictionary to hold presence/absence for this sample
        sample_data = {'Strain': sample}

        # Check presence of each family based on qseqid in t3e_df and RefSeq_ID in T3E_Guide
        for family in families:
            # Get all RefSeq_IDs corresponding to this family
            family_refseq_ids = [refseq for refseq, fam in refseq_to_family.items() if fam == fam

            # Check if any qseqid in t3e_df is in the list of RefSeq_IDs for this family
            if any(t3e_df['qseqid'].isin(family_refseq_ids)):
                sample_data[family] = 'Present'
            else:
                sample_data[family] = 'Absent'

        # Add the sample's data to the matrix
        matrix.append(sample_data)
```

```
# Step 6: Convert the matrix (list of dicts) to a DataFrame and write to CSV
matrix_df = pd.DataFrame(matrix)

# Step 7: Drop columns where all rows have the value 'Absent'
# This works by selecting columns that have any value other than 'Absent'
matrix_df = matrix_df.loc[:, (matrix_df != 'Absent').any(axis=0)]

# Write the matrix to a CSV file
matrix_df.to_csv(output_matrix_file, index=False)

print(f'Matrix written to {output_matrix_file}')
```

5.6.9 BLAST_Matrix_RefSeqID.method2.py

```
import os
import pandas as pd

# File paths
t3e_guide_file = '/Volumes/AS_Postdoc/Postdoc2024/T3SS_Effectors/T3SS_Xanthomonas_Refe
compare_dir = '/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5_Analys
output_matrix_file = '/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5

# Step 1: Read T3E_Guide.csv to get the RefSeq_IDs (these will be the columns)
t3e_guide_df = pd.read_csv(t3e_guide_file)
refseq_ids = sorted(t3e_guide_df['RefSeq_ID'].unique())  # Use RefSeq_ID for columns

# Step 2: Initialize an empty matrix (list of dicts) where each sample will be a row
matrix = []

# Step 3: Loop through each sample's .T3E.csv file in the Compare_BLASTp_and_tBLASTn d
for file in os.listdir(compare_dir):
    if file.endswith('.csv') and not file.startswith('._'):
        sample = file.split('.')[0]  # Extract sample name from the file
        file_path = os.path.join(compare_dir, file)

        # Read the .T3E.csv file
        t3e_df = pd.read_csv(file_path, encoding='utf-8')

        # Check if expected columns are present
        if 'qseqid' not in t3e_df.columns or 'sseqid' not in t3e_df.columns:
            print(f"Warning: Missing expected columns in {file_path}. Skipping file.")
            continue

        # Step 4: Initialize a dictionary to hold presence/absence for this sample
```

```
        sample_data = {'Strain': sample}

        # Check presence of each RefSeq_ID based on sseqid in t3e_df
        for refseq_id in refseq_ids:
            # Check if sseqid in t3e_df matches the current refseq_id
            if any(t3e_df['qseqid'] == refseq_id):
                sample_data[refseq_id] = 'Present'
            else:
                sample_data[refseq_id] = 'Absent'

        # Add the sample's data to the matrix
        matrix.append(sample_data)

# Step 5: Convert the matrix (list of dicts) to a DataFrame
matrix_df = pd.DataFrame(matrix)

# Step 6: Drop columns where all rows have the value 'Absent'
matrix_df = matrix_df.loc[:, (matrix_df != 'Absent').any(axis=0)]

# Step 7: Write the matrix to a CSV file
matrix_df.to_csv(output_matrix_file, index=False)

print(f'Matrix written to {output_matrix_file} with RefSeq_ID columns and empty columns removed.'
```

5.6.10 Filter_tBLASTn.v3.py

```
import os
import pandas as pd

# Define the root directory containing all the subdirectories with Sample.txt files
root_dir = '/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5_Analyses/Effectors/

# Define the column headers
columns = ['qseqid', 'sseqid', 'pident', 'length', 'mismatch', 'gapopen',
           'qstart', 'qend', 'sstart', 'send', 'evalue', 'bitscore', 'qlen', 'slen']

# Loop through each subdirectory in the root directory
for subdir in os.listdir(root_dir):
    subdir_path = os.path.join(root_dir, subdir)

    # Check if it's a directory
    if os.path.isdir(subdir_path):
        sample_file_path = os.path.join(subdir_path, f'{subdir}.txt')
```

```
        # Check if the Sample.txt file exists in the subdirectory
        if os.path.isfile(sample_file_path):
            # Read the tab-delimited file without headers
            df = pd.read_csv(sample_file_path, sep='\t', header=None, names=columns)

            # Apply the first filter: evalue $\leq 1e-50$
            filter1 = df['evalue'] <= 1e-50

            # Apply both filters
            filtered_df = df[filter1]

            # Define the output file path
            output_file_path = os.path.join(subdir_path, f'{subdir}.tBLASTn.filtered.v3

            # Write the filtered data to a new file with headers
            filtered_df.to_csv(output_file_path, sep='\t', index=False, header=True)

print("Filtering complete for all samples.")
```

5.6.11 BLAST_Matrix_FamilyName.v2.py

```
import os
import pandas as pd

# File paths
t3e_guide_file = '/Volumes/AS_Postdoc/Postdoc2024/T3SS_Effectors/T3SS_Xanthomonas_Refer
compare_dir = '/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5_Analys
output_matrix_file = '/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5

# Step 1: Read T3E_Guide.csv to create a dictionary mapping RefSeq_ID to Family
t3e_guide_df = pd.read_csv(t3e_guide_file)
refseq_to_family = dict(zip(t3e_guide_df['RefSeq_ID'], t3e_guide_df['Family']))

# Step 2: Get a list of all effector families (for matrix columns)
families = sorted(t3e_guide_df['Family'].unique())

# Step 3: Initialize an empty matrix (list of dicts) where each sample will be a row
matrix = []

# Step 4: Loop through each sample's .T3E.csv file in the Compare_BLASTp_and_tBLASTn d
for file in os.listdir(compare_dir):
    if file.endswith('.csv') and not file.startswith('._'):
        sample = file.split('.')[0]  # Extract sample name from the file
        file_path = os.path.join(compare_dir, file)
```

```
        # Read the .T3E.csv file
        t3e_df = pd.read_csv(file_path)

        # Step 5: Initialize a dictionary to hold presence/absence for this sample
        sample_data = {'Strain': sample}

        # Check presence of each family based on qseqid in t3e_df and RefSeq_ID in T3E_Guide
        for family in families:
            # Get all RefSeq_IDs corresponding to this family
            family_refseq_ids = [refseq for refseq, fam in refseq_to_family.items() if fam == fam

            # Check if any qseqid in t3e_df is in the list of RefSeq_IDs for this family
            if any(t3e_df['sseqid'].isin(family_refseq_ids)):
                sample_data[family] = 'Present'
            else:
                sample_data[family] = 'Absent'

        # Add the sample's data to the matrix
        matrix.append(sample_data)

# Step 6: Convert the matrix (list of dicts) to a DataFrame and write to CSV
matrix_df = pd.DataFrame(matrix)

# Step 7: Drop columns where all rows have the value 'Absent'
# This works by selecting columns that have any value other than 'Absent'
matrix_df = matrix_df.loc[:, (matrix_df != 'Absent').any(axis=0)]

# Write the matrix to a CSV file
matrix_df.to_csv(output_matrix_file, index=False)

print(f'Matrix written to {output_matrix_file}')
```

5.6.12 BLAST_Matrix_RefSeqID.v2.py

```
import os
import pandas as pd

# File paths
t3e_guide_file = '/Volumes/AS_Postdoc/Postdoc2024/T3SS_Effectors/T3SS_Xanthomonas_Reference_Effec
compare_dir = '/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5_Analyses/Effecto
output_matrix_file = '/Volumes/AS_Postdoc/Postdoc2024/Assembly_Workflow_PlatanusB/Step5_Analyses/

# Step 1: Read T3E_Guide.csv to get the RefSeq_IDs (these will be the columns)
t3e_guide_df = pd.read_csv(t3e_guide_file)
refseq_ids = sorted(t3e_guide_df['RefSeq_ID'].unique())  # Use RefSeq_ID for columns
```

```
# Step 2: Initialize an empty matrix (list of dicts) where each sample will be a row
matrix = []

# Step 3: Loop through each sample's .T3E.csv file in the Compare_BLASTp_and_tBLASTn d
for file in os.listdir(compare_dir):
    if file.endswith('.csv') and not file.startswith('._'):
        sample = file.split('.')[0]  # Extract sample name from the file
        file_path = os.path.join(compare_dir, file)

        # Read the .T3E.csv file
        t3e_df = pd.read_csv(file_path, encoding='utf-8')

        # Check if expected columns are present
        if 'qseqid' not in t3e_df.columns or 'sseqid' not in t3e_df.columns:
            print(f"Warning: Missing expected columns in {file_path}. Skipping file.")
            continue

        # Step 4: Initialize a dictionary to hold presence/absence for this sample
        sample_data = {'Strain': sample}

        # Check presence of each RefSeq_ID based on sseqid in t3e_df
        for refseq_id in refseq_ids:
            # Check if sseqid in t3e_df matches the current refseq_id
            if any(t3e_df['qseqid'] == refseq_id):
                sample_data[refseq_id] = 'Present'
            else:
                sample_data[refseq_id] = 'Absent'

        # Add the sample's data to the matrix
        matrix.append(sample_data)

# Step 5: Convert the matrix (list of dicts) to a DataFrame
matrix_df = pd.DataFrame(matrix)

# Step 6: Drop columns where all rows have the value 'Absent'
matrix_df = matrix_df.loc[:, (matrix_df != 'Absent').any(axis=0)]

# Step 7: Write the matrix to a CSV file
matrix_df.to_csv(output_matrix_file, index=False)

print(f'Matrix written to {output_matrix_file} with RefSeq_ID columns and empty columns
```

Bibliography

Assis, R. A. B., Varani, A. M., Sagawa, C. H. D., Patané, J. S. L., Setubal, J. C., Uceda-Campos, G., da Silva, A. M., Zaini, P. A., Almeida, N. F., Moreira, L. M., and Dandekar, A. M. (2021). A comparative genomic analysis of xanthomonas arboricola pv. juglandis strains reveal hallmarks of mobile genetic elements in the adaptation and accelerated evolution of virulence. *Genomics*, 113:2513–2525.

Behlau, F., Canteros, B. I., Minsavage, G. V., Jones, J. B., and Graham, J. H. (2011). Molecular characterization of copper resistance genes from xanthomonas citri subsp. citri and xanthomonas alfalfae subsp. citrumelonis. *Applied and environmental microbiology*, 77(12):4089–4096.

Behlau, F., Fonseca, A., and Belasque Jr, J. (2016). A comprehensive analysis of the asiatic citrus canker eradication programme in são paulo state, brazil, from 1999 to 2009. *Plant Pathology*, 65(8):1390–1399.

Borowiec, M. L. (2016). Amas: a fast tool for alignment manipulation and computing of summary statistics. *PeerJ*, 4:e1660.

Chen, S., Zhou, Y., Chen, Y., and Gu, J. (2018). fastp: an ultra-fast all-in-one fastq preprocessor. *Bioinformatics*, 34:i884–i890.

Croucher, N. J., Page, A. J., Connor, T. R., Delaney, A. J., Keane, J. A., Bentley, S. D., Parkhill, J., and Harris, S. R. (2015). Rapid phylogenetic analysis of large samples of recombinant bacterial whole genome sequences using gubbins. *Nucleic Acids Research*, 43:e15–e15.

Cuesta-Morrondo, S., Redondo, C., Palacio-Bielsa, A., Garita-Cambronero, J., and Cubero, J. (2022). Complete genome sequence resources of six strains of the most virulent pathovars of xanthomonas arboricola using long- and short-read sequencing approaches. *Phytopathology*, 112:1808–1813.

Didelot, X., Croucher, N. J., Bentley, S. D., Harris, S. R., and Wilson, D. J. (2018). Bayesian inference of ancestral dates on bacterial phylogenetic trees. *Nucleic Acids Research*, 46:e134.

Didelot, X. and Wilson, D. J. (2015). Clonalframeml: Efficient inference of recombination in whole bacterial genomes. *PLoS Comput Biol*, 11:e1004041.

Drummond, A. J., Pybus, O. G., Rambaut, A., Forsberg, R., and Rodrigo, A. G. (2003). Measurably evolving populations. *Trends in Ecology and Evolution*, 18(9):481–488.

Duchene, S., Lemey, P., Stadler, T., Ho, S. Y. W., Duchene, D. A., Dhanasekaran, V., and Baele, G. (2020). Bayesian evaluation of temporal signal in measurably evolving populations. *Molecular Biology and Evolution*, 37(11):3363–3379.

Fernandes, C., Martins, L., Teixeira, M., Blom, J., Pothier, J. F., Fonseca, N. A., and Tavares, F. (2021). Comparative genomics of xanthomonas euroxanthea and xanthomonas arboricola pv. juglandis strains isolated from a single walnut host tree. *Microorganisms*, 9(3):624.

Gautreau, G., Bazin, A., Gachet, M., Planel, R., Burlot, L., Dubois, M., Perrin, A., Médigue, C., Calteau, A., Cruveiller, S., Matias, C., Ambroise, C., Rocha, E. P. C., and Vallenet, D. (2020). Ppanggolin: Depicting microbial diversity via a partitioned pangenome graph. *PLOS Computational Biology*, 16(e1007732).

Giovanardi, D., Bonneau, S., Gironde, S., Saux, M. F.-L., Manceau, C., and Stefani, E. (2016). Morphological and genotypic features of xanthomonas arboricola pv. juglandis populations from walnut groves in romagna region, italy. *European Journal of Plant Pathology*, 145:1–16.

Goluch, T., Bogdanowicz, D., and Giaro, K. (2020). Visual treecmp: Comprehensive comparison of phylogenetic trees on the web. *Methods in Ecology and Evolution*, 11:494–499.

Gurevich, A., Saveliev, V., Vyahhi, N., and Tesler, G. (2013). Quast: quality assessment tool for genome assemblies. *Bioinformatics*, 29:1072–1075.

Jombart, T. (2008). adegenet: a r package for the multivariate analysis of genetic markers. *Bioinformatics*, 24:1403–1405.

Kajitani, R., Yoshimura, D., Ogura, Y., Gotoh, Y., Hayashi, T., and Itoh, T. (2020). Platanus_b: an accurate de novo assembler for bacterial genomes using an iterative error-removal process. *DNA Research*, 27:dsaa014.

Kalyaanamoorthy, S., Minh, B. Q., Wong, T. K. F., von Haeseler, A., and Jermiin, L. S. (2017). Modelfinder: fast model selection for accurate phylogenetic estimates. *Nat Methods*, 14:587–589.

Kille, B., Nute, M. G., Huang, V., Kim, E., Phillippy, A. M., and Treangen, T. J. (2024). Parsnp 2.0: scalable core-genome alignment for massive microbial datasets. *Bioinformatics*, 40:btae311.

Kolmogorov, M., Raney, B., Paten, B., and Pham, S. (2014). Ragout—a reference-assisted assembly tool for bacterial genomes. *Bioinformatics*, 30:i302–i309.

Lee, Y., Hendson, M., Panopoulos, N., and Schroth, M. (1994). Molecular cloning, chromosomal mapping, and sequence analysis of copper resistance genes from xanthomonas campestris pv. juglandis: homology with small blue copper proteins and multicopper oxidase. *Journal of bacteriology*, 176(1):173–188.

Li, H., Handsaker, B., Wysoker, A., Fennell, T., Ruan, J., Homer, N., Marth, G., Abecasis, G., Durbin, R., and Subgroup, . G. P. D. P. (2009). The sequence alignment/map format and samtools. *Bioinformatics*, 25:2078–2079.

Marin, T., Galvanin, A., Lanza, F., and Behlau, F. (2019). Description of copper tolerant xanthomonas citri subsp. citri and genotypic comparison with sensitive and resistant strains. *Plant Pathology*, 68(6):1088–1098.

Minh, B. Q., Schmidt, H. A., Chernomor, O., Schrempf, D., Woodhams, M. D., von Haeseler, A., and Lanfear, R. (2020). Iq-tree 2: New models and efficient methods for phylogenetic inference in the genomic era. *Molecular Biology and Evolution*, 37:1530–1534.

Parks, D. H., Imelfort, M., Skennerton, C. T., Hugenholtz, P., and Tyson, G. W. (2015). Checkm: assessing the quality of microbial genomes recovered from isolates, single cells, and metagenomes. *Genome Res.*, 25:1043–1055.

Rambaut, A., Lam, T. T., Max Carvalho, L., and Pybus, O. G. (2016). Exploring the temporal structure of heterochronous sequences using tempest (formerly path-o-gen). *Virus Evolution*, 2:vew007.

Richard, D., Ravigné, V., Rieux, A., Facon, B., Boyer, C., Boyer, K., Grygiel, P., Javegny, S., Terville, M., Canteros, B. I., et al. (2017). Adaptation of genetically monomorphic bacteria: evolution of copper resistance through multiple horizontal gene transfers of complex and versatile mobile genetic elements. *Molecular Ecology*, 26(7):2131–2149.

Scortichini, M., Marchesi, U., and Di Prospero, P. (2001). Genetic diversity of xanthomonas arboricola pv. juglandis (synonyms: X. campestris pv. juglandis; x. juglandis pv. juglandis) strains from different geographical areas shown by repetitive polymerase chain reaction genomic fingerprinting. *Journal of Phytopathology*, 149:325–332.

Subedi, A., Barrera, L. B. T. d. l., Ivey, M. L., Egel, D. S., Kebede, M., Kara, S., Aysan, Y., Minsavage, G. V., Roberts, P. D., Jones, J. B., and Goss, E. M. (2024). Population genomics reveals an emerging lineage of xanthomonas perforans on pepper. *Phytopathology*, 114:241–250.

Tatusova, T., DiCuccio, M., Badretdin, A., Chetvernin, V., Nawrocki, E. P., Zaslavsky, L., Lomsadze, A., Pruitt, K. D., Borodovsky, M., and Ostell, J. (2016). Ncbi prokaryotic genome annotation pipeline. *Nucleic Acids Research*, 44:6614–6624.

Tonkin-Hill, G., Lees, J. A., Bentley, S. D., Frost, S. D., and Corander, J. (2018). Rhierbaps: An r implementation of the population clustering algorithm hierbaps. *Wellcome Open Research*, 3:93.

Treangen, T. J., Ondov, B. D., Koren, S., and Phillippy, A. M. (2014). The harvest suite for rapid core-genome alignment and visualization of thousands of intraspecific microbial genomes. *Genome Biol*, 15:524.

Vasimuddin, M., Misra, S., Li, H., and Aluru, S. (2019). Efficient architecture-aware acceleration of bwa-mem for multicore systems. In *2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 314–324.

Walker, B. J., Abeel, T., Shea, T., Priest, M., Abouelliel, A., Sakthikumar, S., Cuomo, C. A., Zeng, Q., Wortman, J., Young, S. K., and Earl, A. M. (2014). Pilon: An integrated tool for comprehensive microbial variant detection and genome assembly improvement. *PLoS ONE*, 9:e112963.